JIUM

지음(JIUM)

목조 건축 프로젝트



협동-2: AI(Computer Vision)기반 협동 로봇 작업 어시스턴트 구현 프로젝트

D - 02

조장. 곽준영

조원. 김동국

조원. 박경모

조원. 이동현



CONTENTS

01 프로젝트 개요

02 프로젝트 팀 구성 및 역할

03 프로젝트 수행 절차 및 방법

04 프로젝트 수행 경과

05 자체 평가 의견

JIUM

01.
프로젝트 개요

- 프로젝트 주제 및 선정 배경
- 프로젝트 내용
- 활용방안 및 기대 효과



주제 선정





JIUM

시장 및 산업 동향

약 7조 1,000억원
목표 시장의 크기

글로벌 다관절 로봇 팔
및 정밀 시공 시장



약 10조 5,000억원
전체 시장의 크기

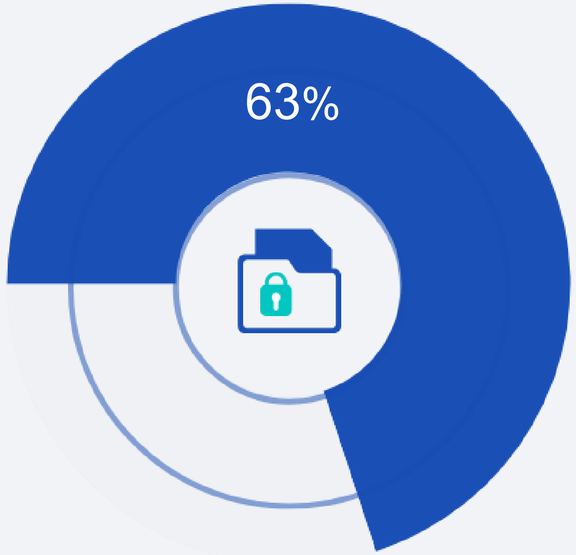
글로벌 건설 로봇 시장

약 5,000억원
초기진입시장의 크기

국내 협동 로봇(Cobot) 기반
스마트 건설 시장

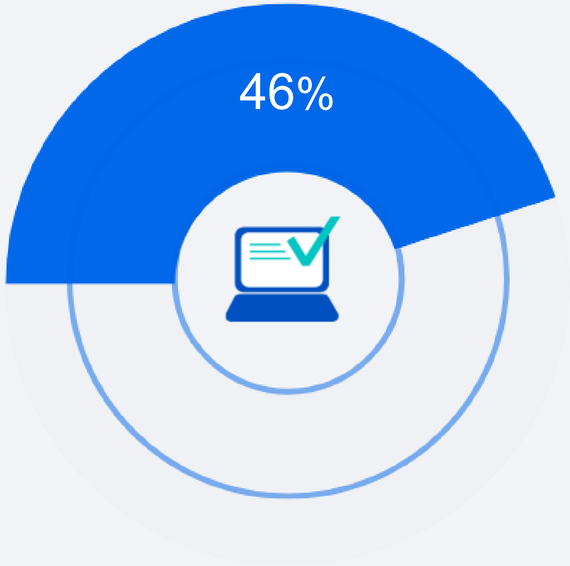
JIUM

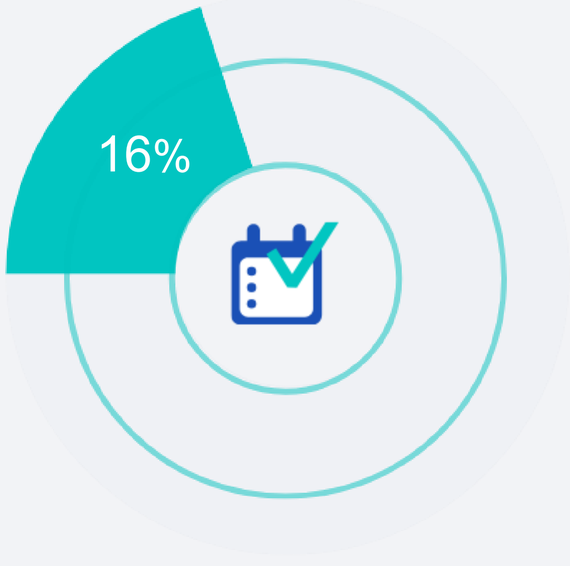
위험도 분석



평균 연령의 급증

고령층 비중이 전체의 : 62.8%
건설 근로자의 평균 연령 : 51.8세





안전 사고 위험

전체 산업 재해 중 업종별 사망사고 1위
전체 산업재해 사망자 중 건설업 : 46.4%
신체 부담 작업으로 인한 질병 재해율 : 15.7%

프로젝트 활용방안 및 기대효과

건설 현장의 안전성 및 효율성 극대화

고위험 반복 작업(자재 운반, 정밀 배치)을 협동 로봇 Doosan M0609가 전담 수행함으로써 작업자의 산업재해 및 근골격계 질환 발생률을 획기적으로 낮춥니다.

데이터 기반의 초정밀 건축 구현

YOLO 기반 객체 인식과 3D 공간 좌표 추출 기술을 결합하여, 인간의 눈으로 놓치기 쉬운 오차를 최소화하고 설계도(Blueprint)와 일치하는 정밀 시공을 실현합니다.

친환경 목조건축 시장의 디지털 전환

탄소 중립 시대의 핵심인 목조건축 분야에 자동화 솔루션을 도입하여, 고부가가치 공학 목재(CLT 등)의 시공 경쟁력을 확보하고 친환경 건축 확산에 기여합니다.

사람의 안전과 로봇의 정밀함이 만나는 곳.
협업 건축 솔루션 ‘지음(JIUM)’

프로젝트 개요

1. Blueprint Transmission

사용자가 웹 인터페이스(React 기반)를 통해 건축 설계도(Grid Canvas 데이터)를 전송하면, 시스템은 이를 로봇이 이해할 수 있는 3D 좌표 데이터로 변환합니다.

2. Block Pose-estimation

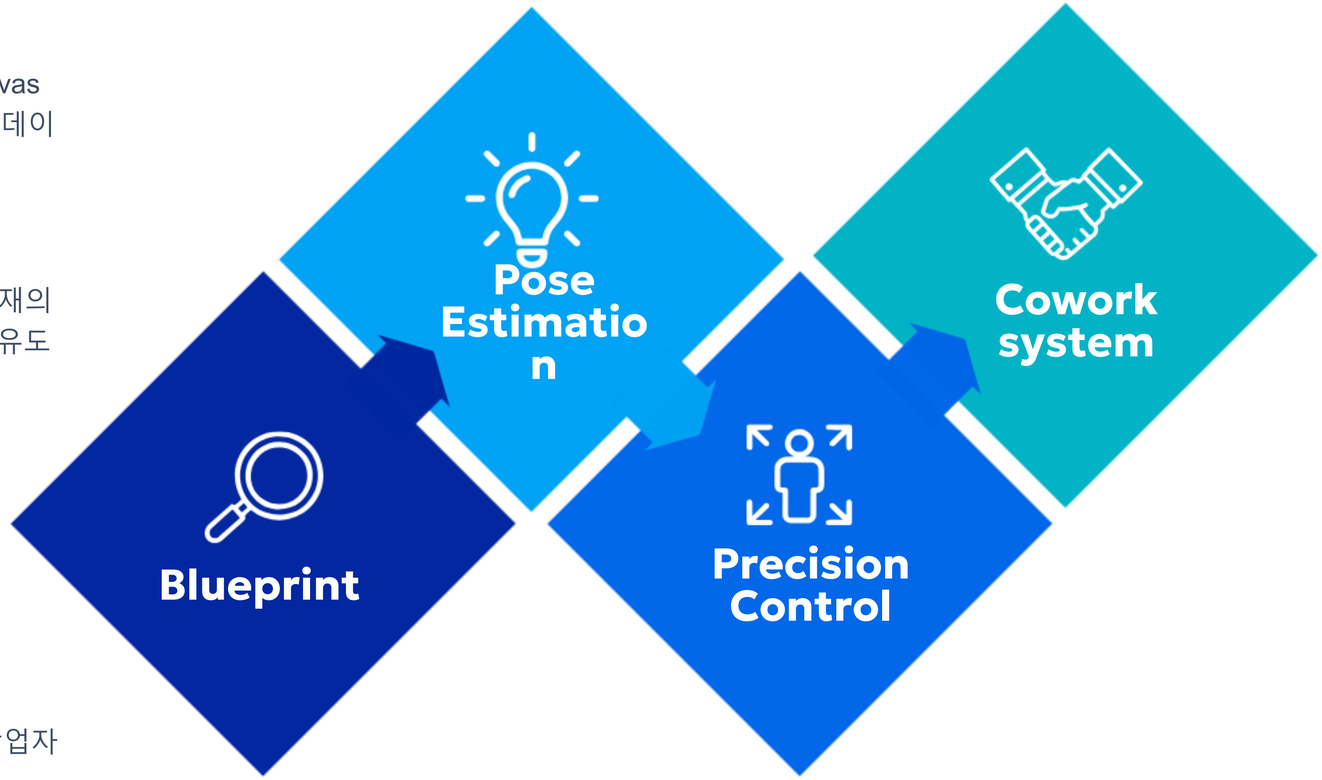
카메라를 통해 현재 작업대 위의 블록들을 인식합니다. YOLO로 부재의 종류를 판별하고, Open3D를 활용해 뎁스 데이터로부터 정밀한 6자유도(6-DoF) 포즈를 추정합니다.

3. Precision Control

ROS 2와 두산 로봇 API를 연동하여 계산된 경로에 따라 블록을 집어 정해진 위치에 배치합니다. 실시간 피드백 루프를 통해 시공 오차를 보정합니다.

4. Co-work System

음성 인식을 통해 작업 현장에서 즉각적인 지시를 내리고, 로봇은 작업자의 위치를 인식하여 안전 거리를 유지하며 협동 작업을 수행합니다.



JIUM

프로젝트 기대효과

High-Precision (초정밀 시공)

비정형 목조 구조물의 정밀 결합이 가능해지며, 시공 결함으로 인한 재작업률을 획기적으로 낮춥니다.

Safety First (작업 안전성)

건설업의 고질적인 문제인 산업재해 및 근골격계 질환을 예방하고, 안전한 스마트 건설 환경을 조성합니다.



Productivity (생산성 혁신)

건설 현장의 만성적인 인력 부족 문제를 해결하고, 동일 시간 대비 건축물 생산량을 30% 이상 향상시킵니다.

Sustainability (지속 가능한 건축)

친환경 목조건축의 대중화를 앞당겨 건설 산업의 탄소 중립(Net-Zero) 달성에 기여합니다.

JIUM

02.
프로젝트 팀 구성 및 역할



프로젝트 추진 팀 소개

파트	이름	담당 업무
Control	곽준영	PM / UI　System Integration　Cowork-system
	김동국	System Integration　M0609 Control　Stop-Recovery
Detection	박경모	YOLO Detection　Pose Estimation　3D modeling
	이동현	Pose Estimation　Docker(ROS2-setup)　Yolo - FoundationPose

JIUM

03.
프로젝트 수행 절차 및 방법

- Schedule Timeline
- Tech Stack
- System Architecture
- Node Architecture
- System Flow



프로젝트 세부 진행 일정

구분	기간	활동	비고
프로젝트 계획 및 환경 구축	2/10~2/12	프로젝트 기획 / Business Requirement 정의 / Timetable 작성	전체 방향성 확정
Detection	2/10~2/24	System Requirement / System Architecture 설계	객체 인식 구현
기본 동작 개발	2/19~2/20	좌표 티칭 / 물리 환경 제작 / 로봇 동작 개발 / ROS 통신	로봇 핵심 로직 충돌·리스크 제거
UI 개발	2/10~2/24	UI / System Monitor & Control	User Interface 모니터링 제어 환경
루프/안전복구 구현	2/24~2/25	완성형 루프 / 일시정지 / 재개 / 안전복구	안전성 및 지속성 구현
전체 시스템 통합	2/24~2/25	전체 Integration	End-to-End 검증
결과물 정리 및 발표	2/25~2/27	설계문서 / 시연 / POC 영상 / PPT	시연·제출 및 발표
총 개발기간	2/10~2/27(총 12일)		

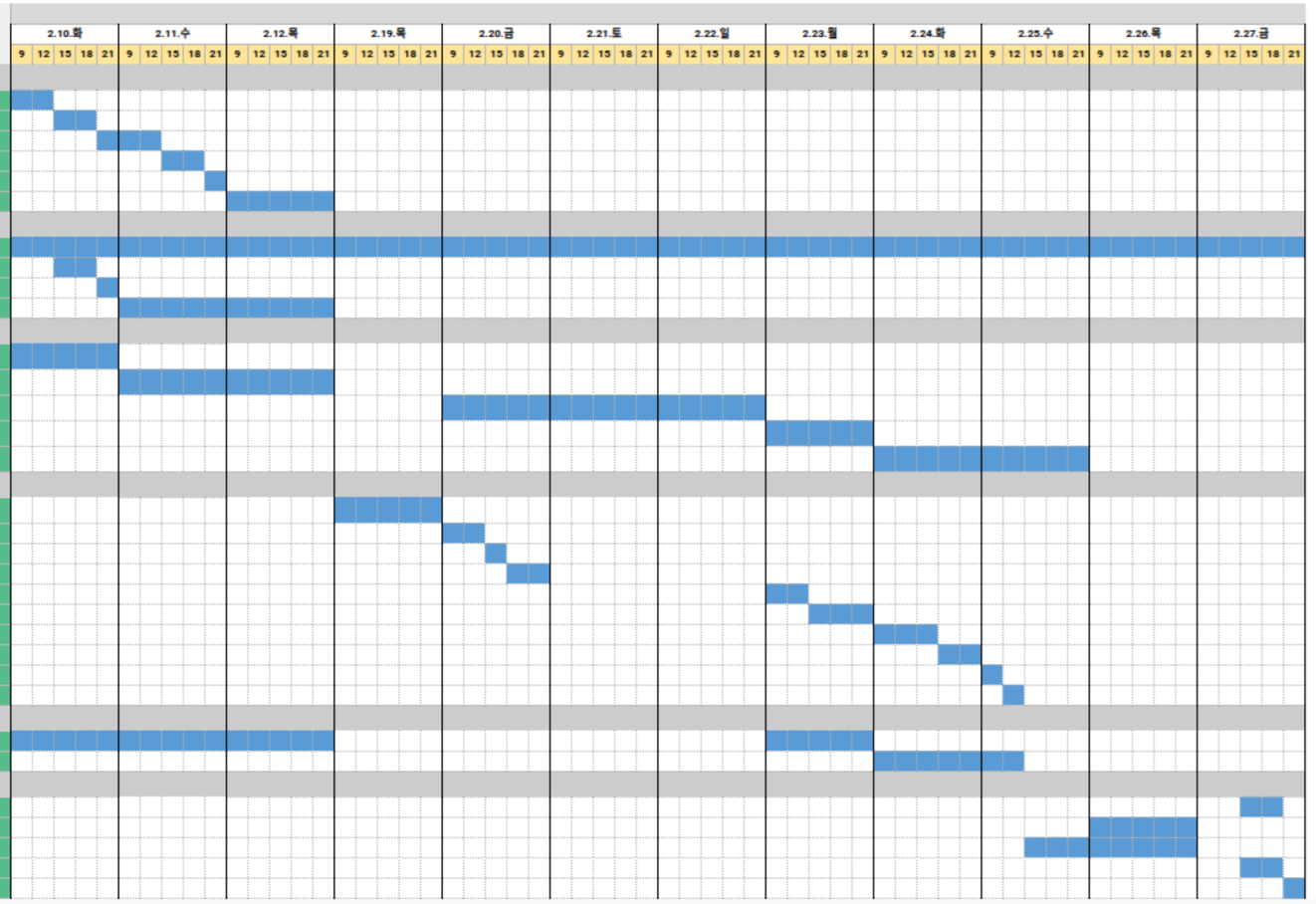
JIUM

프로젝트 세부 진행 일정

Gantt Chart

프로젝트 이름	지음(JIUM)	팀명	D-2
프로젝트 관리자	곽준영,김동국,박경모,이동현	기간	26년02월10일 ~ 26년02월27일

WBS 번호	작업 제목	작업 소유자	시작일	마감일	기간	작업 완료 비율
1	**Project Planning (Team)**					
1.1	Search	Team	26년 2월 10일	26년 2월 10일	1	100
1.2	Business Requirement	Team	26년 2월 10일	26년 2월 10일	1	100
1.3	Flow Chart	Team	26년 2월 10일	26년 2월 11일	2	100
1.4	Timetable	Team	26년 2월 11일	26년 2월 11일	1	100
1.5	Space Planning	Team	26년 2월 11일	26년 2월 11일	1	100
1.6	Technical Setup	Team	26년 2월 12일	26년 2월 12일	1	100
2	**Project Managing**					
2.1	Timetable Manage	Team	26년 2월 10일	26년 2월 27일	12	100
2.2	Business Requirement	Team	26년 2월 10일	26년 2월 10일	1	100
2.3	System Requirement	Team	26년 2월 10일	26년 2월 10일	1	100
2.4	System Architecture	김동국	26년 2월 11일	26년 2월 12일	2	100
3	**Detection**					
3.1	FoundationPose	이동현,박경모	26년 2월 10일	26년 2월 10일	1	100
3.2	Pose Estimation	이동현,박경모	26년 2월 11일	26년 2월 12일	1	100
3.3	동적 타겟팅 구현	이동현,박경모	26년 2월 20일	26년 2월 22일	1	100
3.4	알고리즘 보정 및 안정화	이동현,박경모	26년 2월 23일	26년 2월 23일	1	100
3.5	최종 성능 개선	이동현,박경모	26년 2월 24일	26년 2월 25일	1	100
4	**M0609 CONTROL**					
4.1	Coordinate mapping	김동국	26년 2월 19일	26년 2월 19일	1	100
4.2	개별 기능 테스트(통신 미포함)	김동국,곽준영	26년 2월 20일	26년 2월 20일	1	100
4.3	개별 기능 테스트(통신 포함)	김동국	26년 2월 20일	26년 2월 20일	1	100
4.4	integration 1차(UI제외)	김동국, 이동현	26년 2월 20일	26년 2월 20일	1	100
4.5	integration 2차(UI포함)	김동국, 곽준영	26년 2월 23일	26년 2월 23일	1	100
4.6	완성형 루프 구현	김동국	26년 2월 23일	26년 2월 23일	1	100
4.7	일시정지,재개 구현	김동국	26년 2월 24일	26년 2월 24일	1	100
4.8	안전복구 구현	김동국	26년 2월 24일	26년 2월 24일	1	100
4.9	Code Review	Team	26년 2월 25일	26년 2월 25일	1	100
4.10.	Integration 3차 (전체 통합)	Team	26년 2월 25일	26년 2월 25일	1	100
5	**Monitor/Database**					
5.1	UI	곽준영	26년 2월 10일	26년 2월 23일	4	100
5.2	Integration	곽준영	26년 2월 24일	26년 2월 25일	2	100
6	**Presentation**					
6.1	Demonstration	Team	26년 2월 27일	26년 2월 27일	1	100
6.2	POC Video	Team	26년 2월 26일	26년 2월 26일	1	100
6.3	PPT	Team	26년 2월 25일	26년 2월 26일	2	100
6.4	presentaion	Team	26년 2월 27일	26년 2월 27일	1	100
6.5	Submitting	곽준영	26년 2월 27일	26년 2월 27일	1	100



Tech Stack

Dev

ROS 2

Detection

NVIDIA FoundationPose

ultralytics

MediaPipe

Language

UI

ROS(Foxy/Humble)

Cyclone DDS

rosbridge

HW

M0609







Project Scenario

Docker Setup

HOST





NVIDIA FoundationPose

Docker







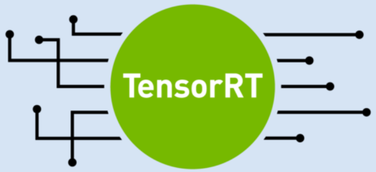




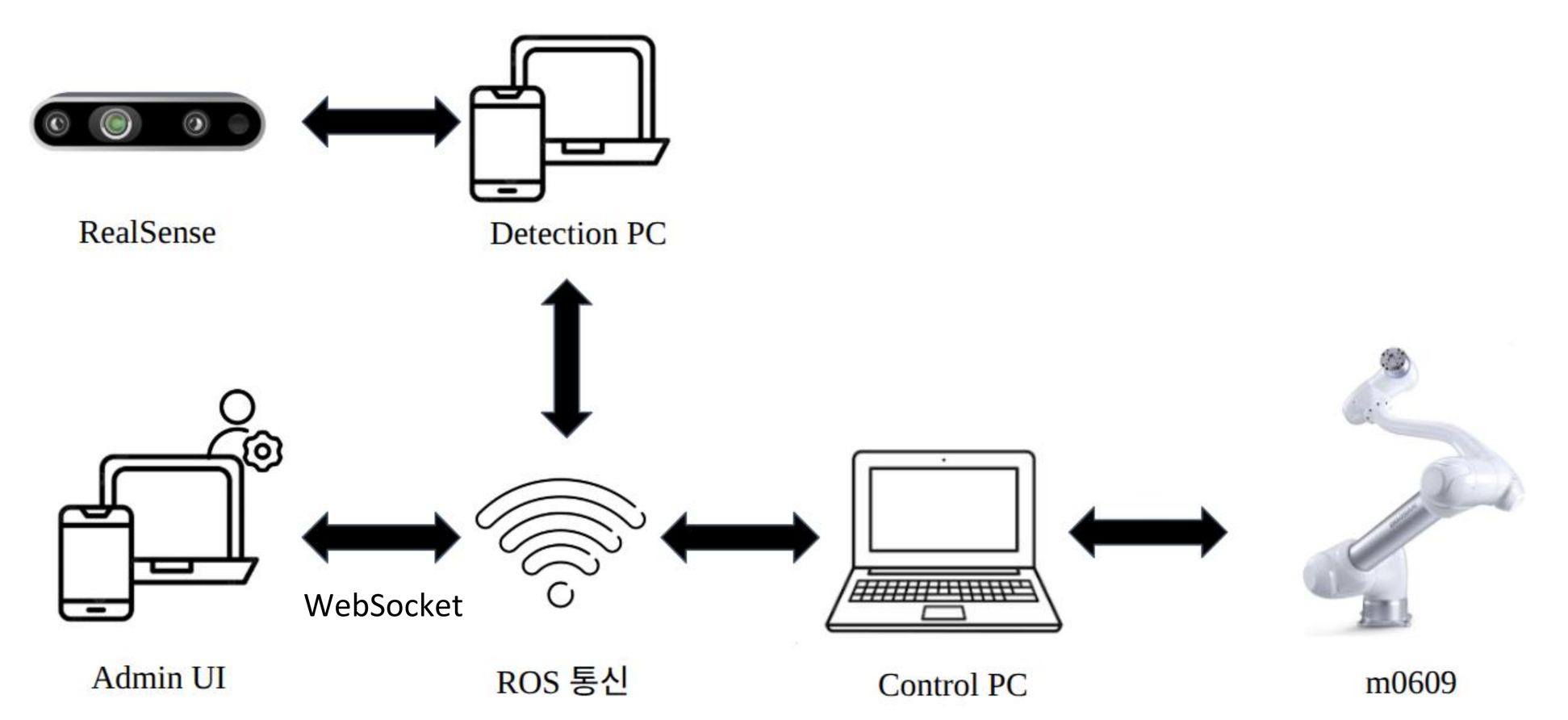




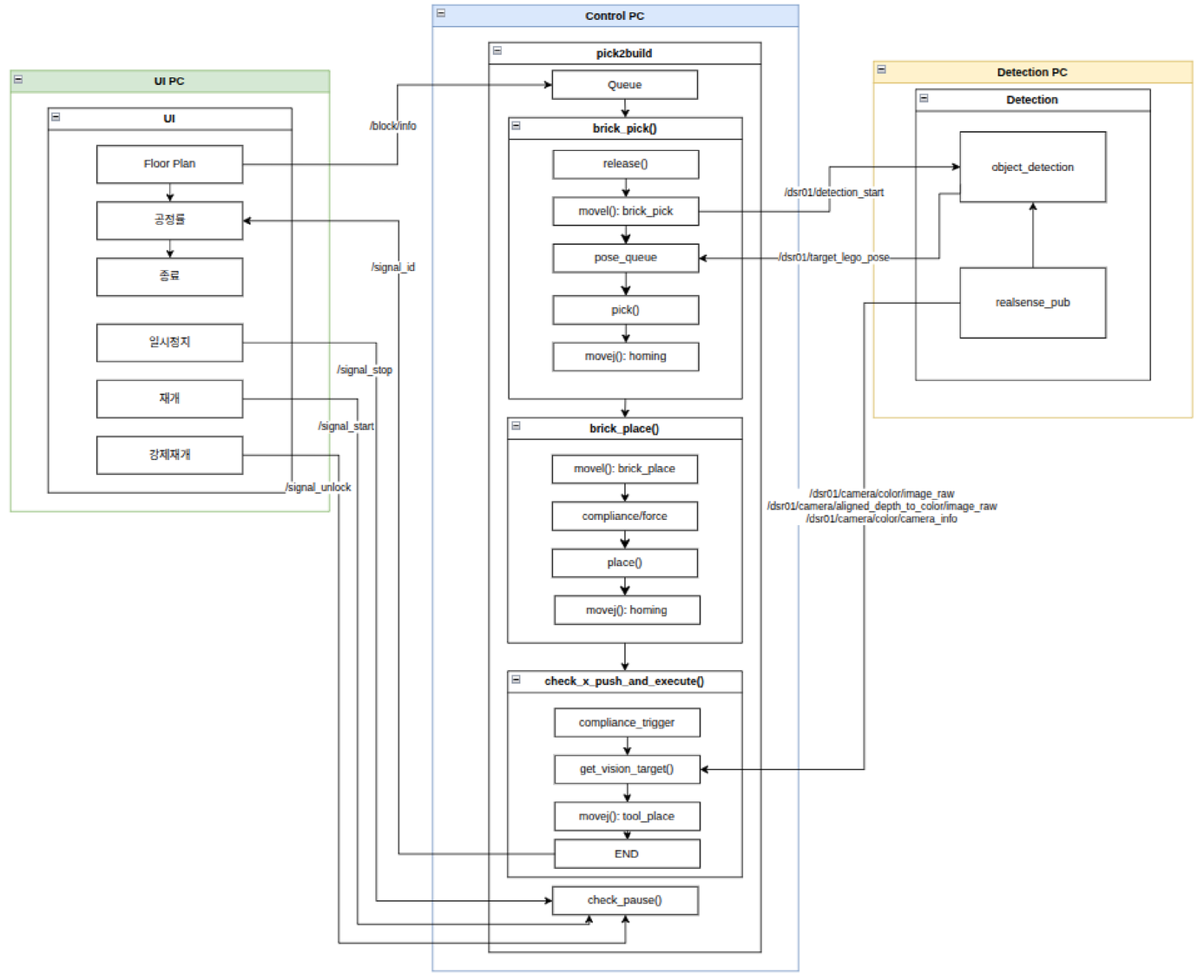




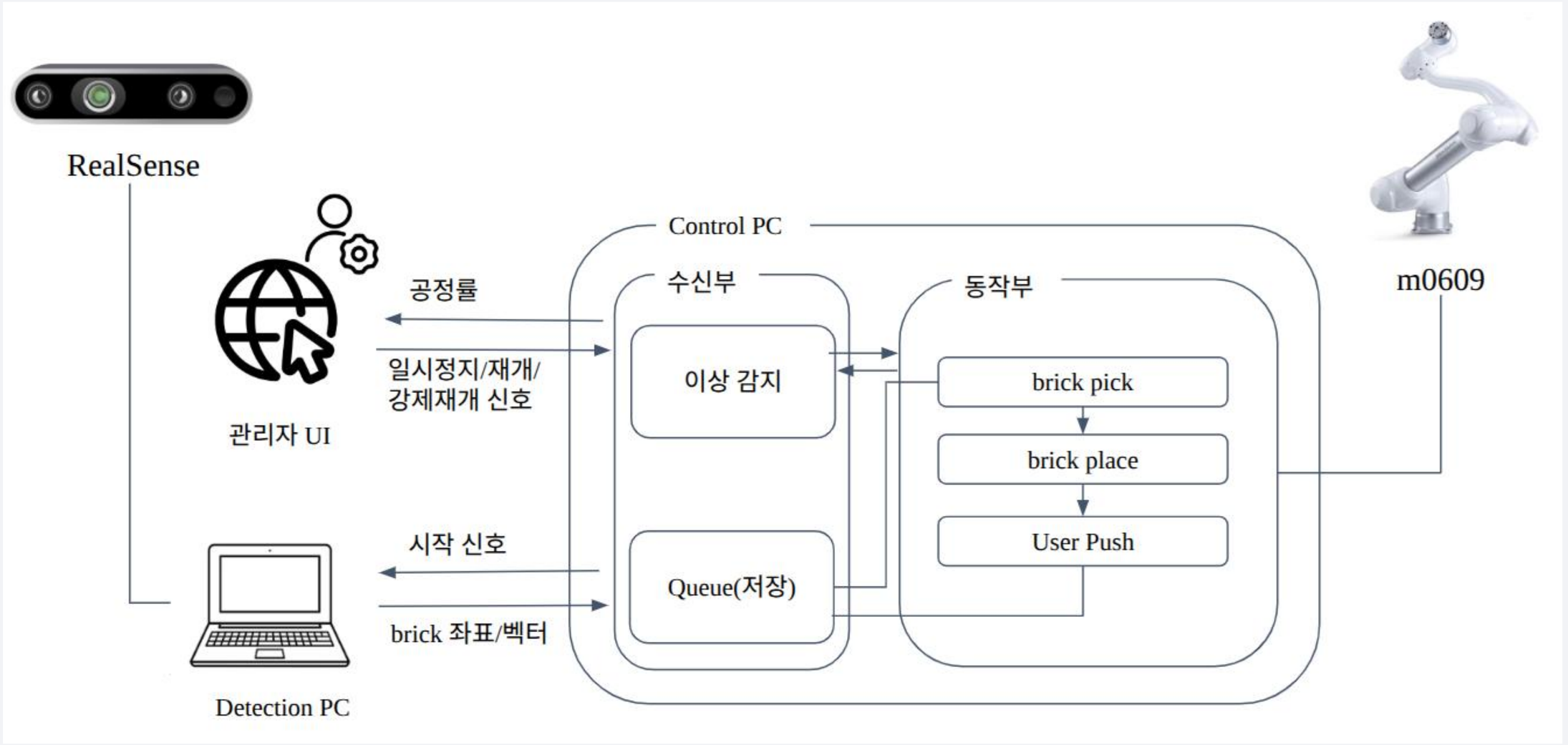
System Architecture



Node Architecture



System Flow


RealSense
관리자 UI
Detection PC
공정률
일시정지/재개/
강제재개 신호
시작 신호
brick 좌표/벡터
Control PC
수신부
이상 감지
Queue(저장)
동작부
brick pick
brick place
User Push
m0609

JIUM

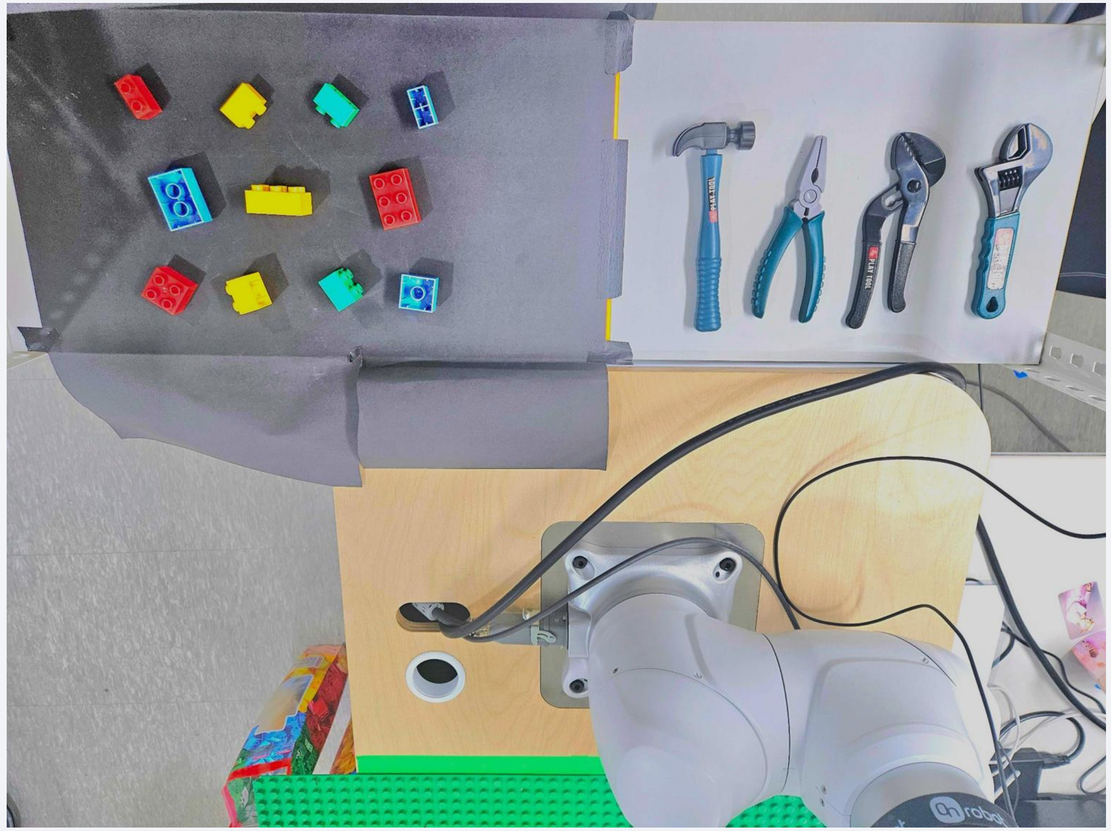
04.
프로젝트 수행 경과

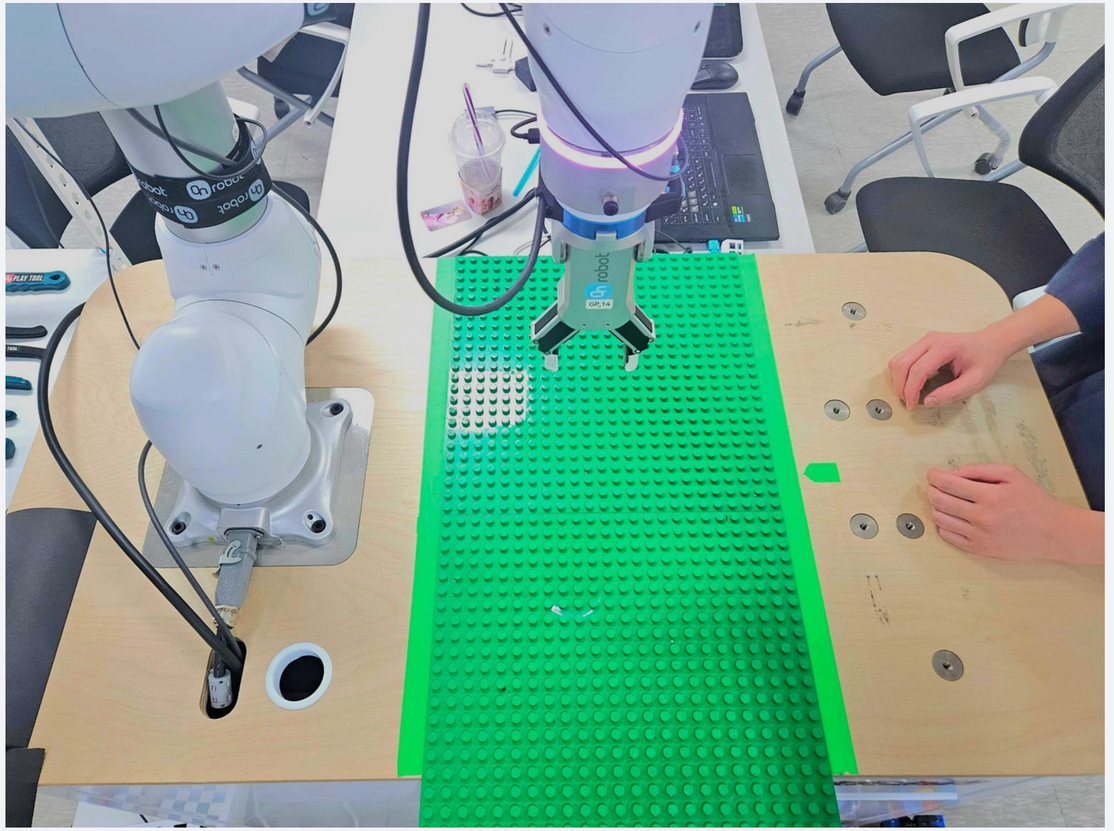
- Project Scenario
- Demonstration video
- UI
- Detection
- Control



Project Scenario

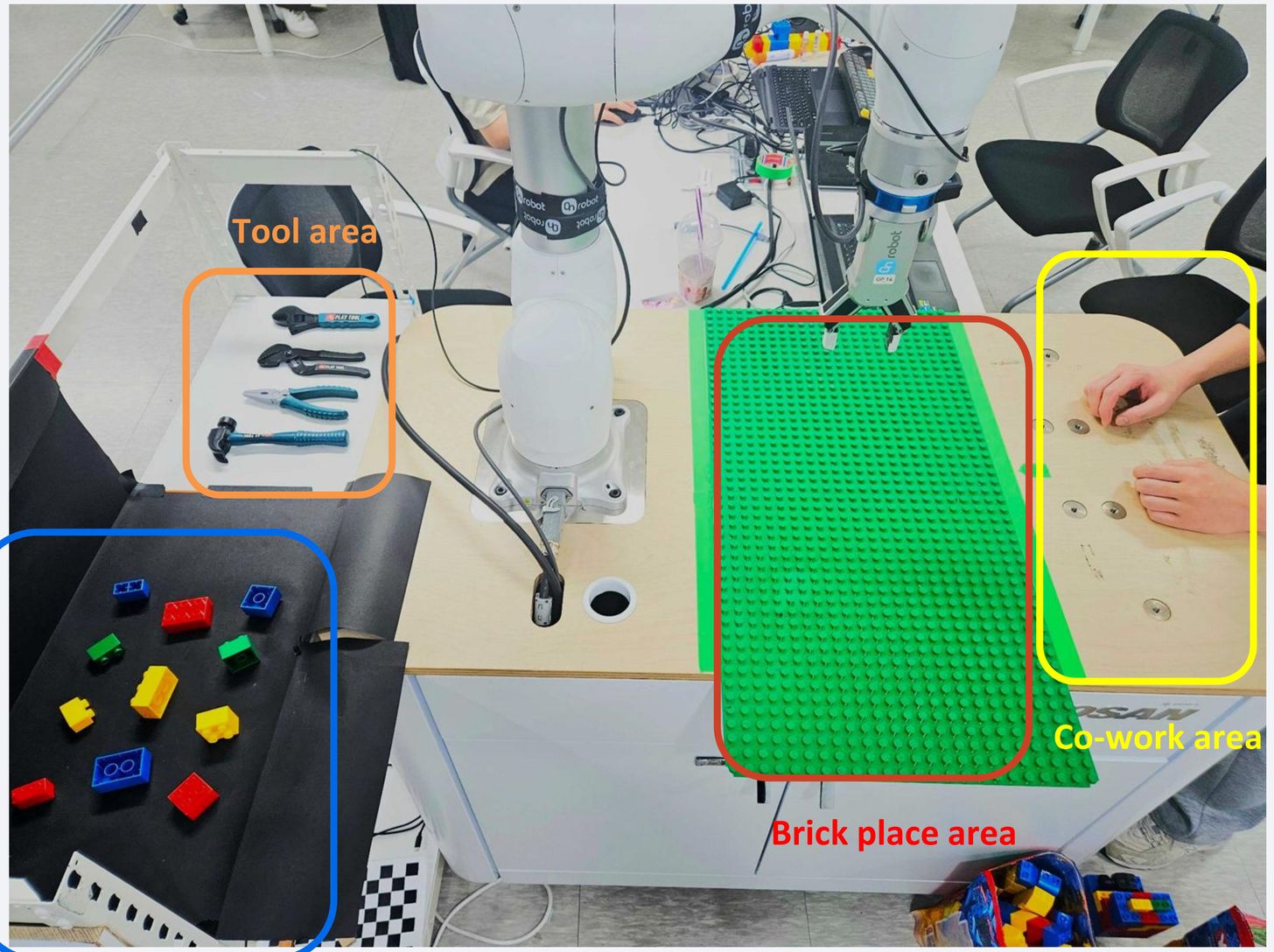
Workspace Setup





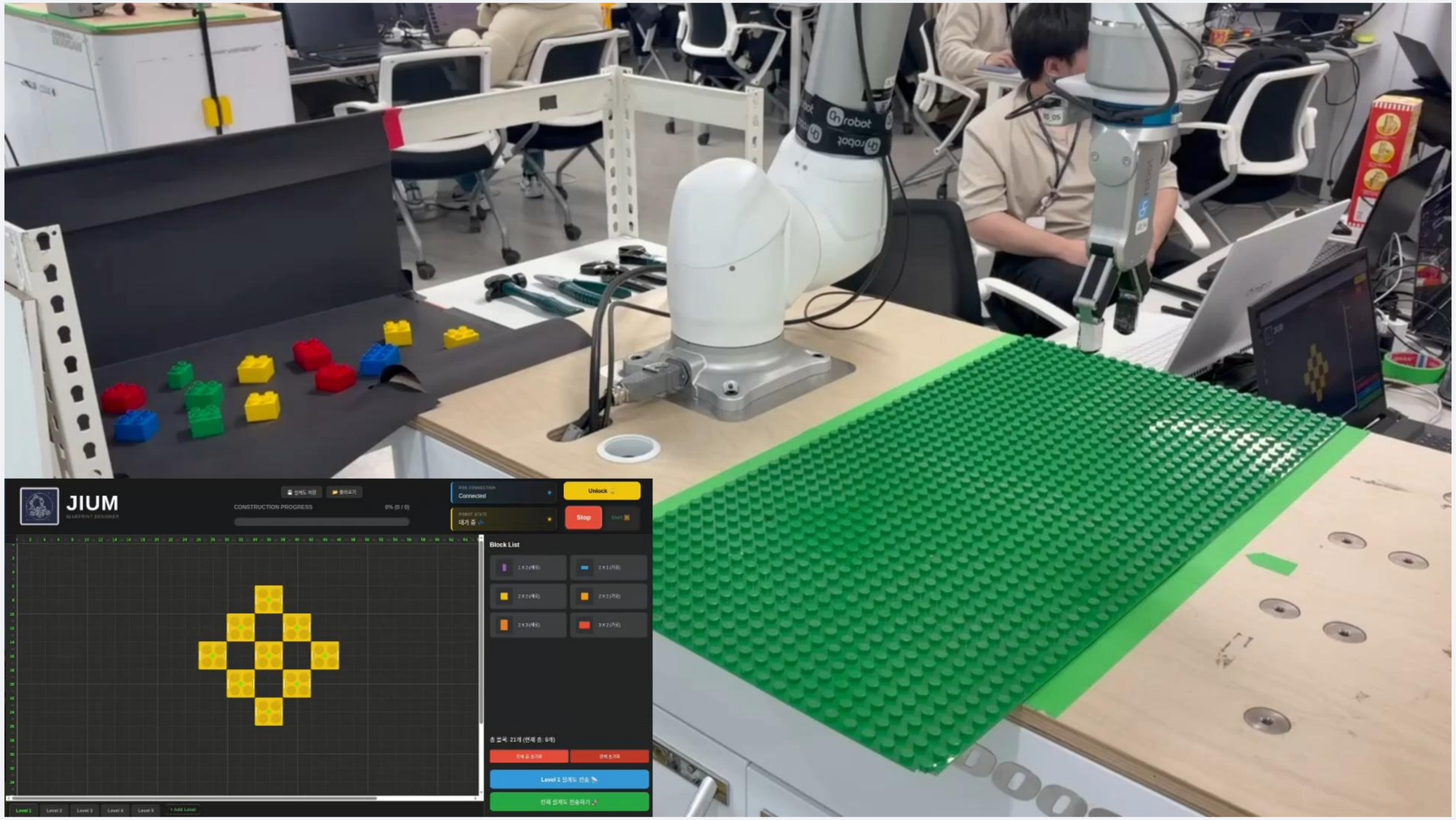
Project Scenario

Workspace Setup



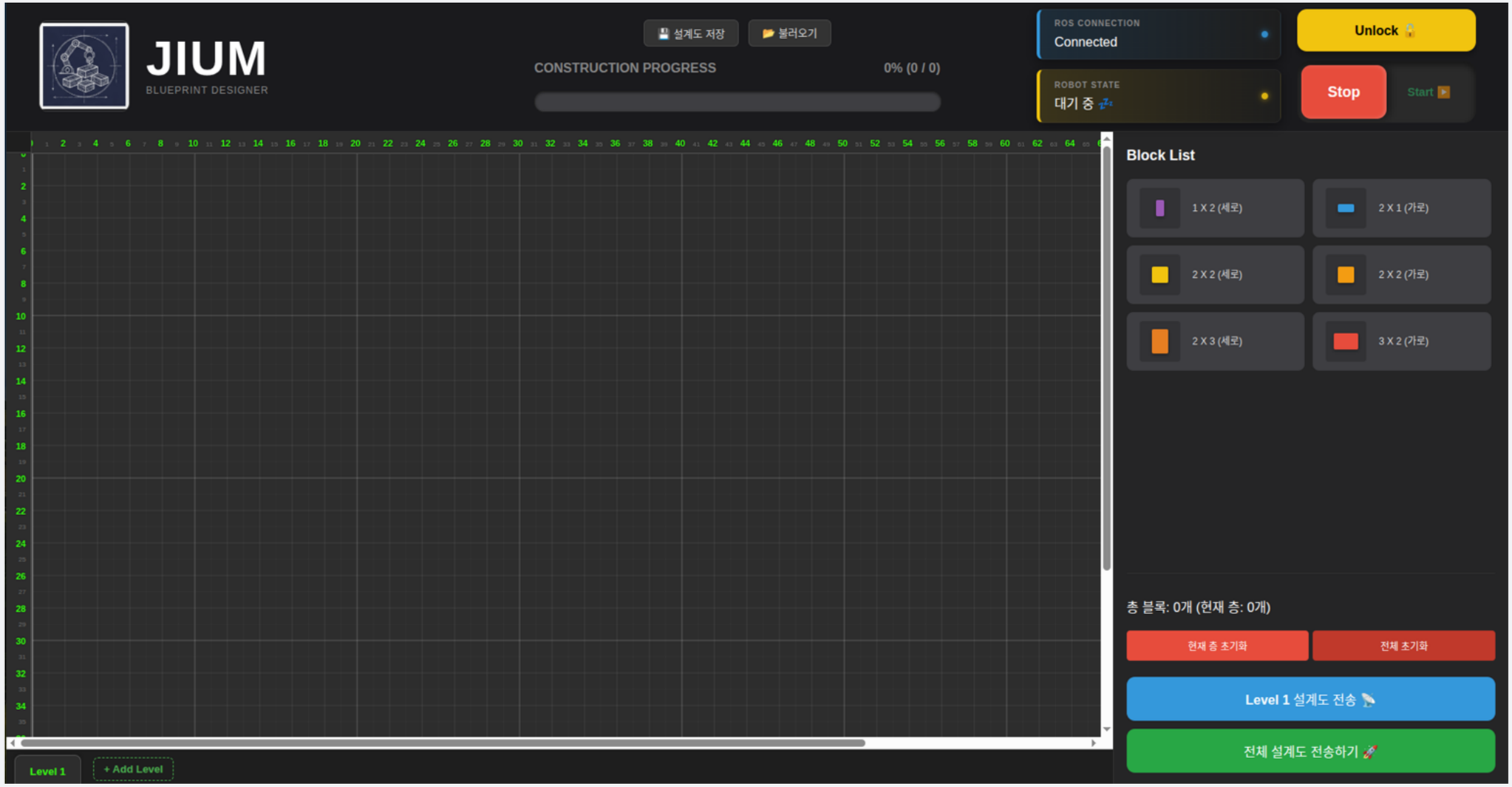
Project Scenario

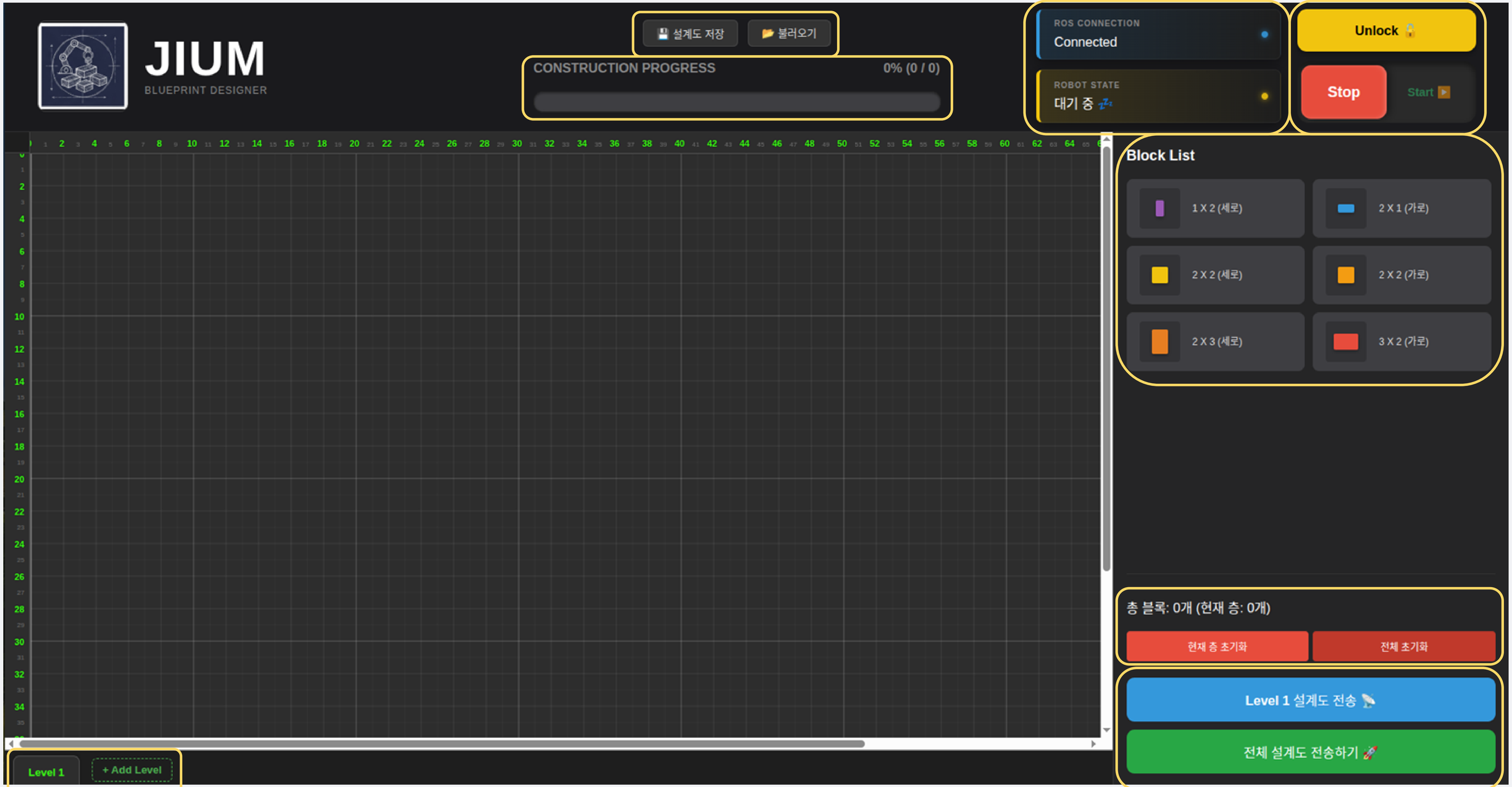
Demonstration video



UI

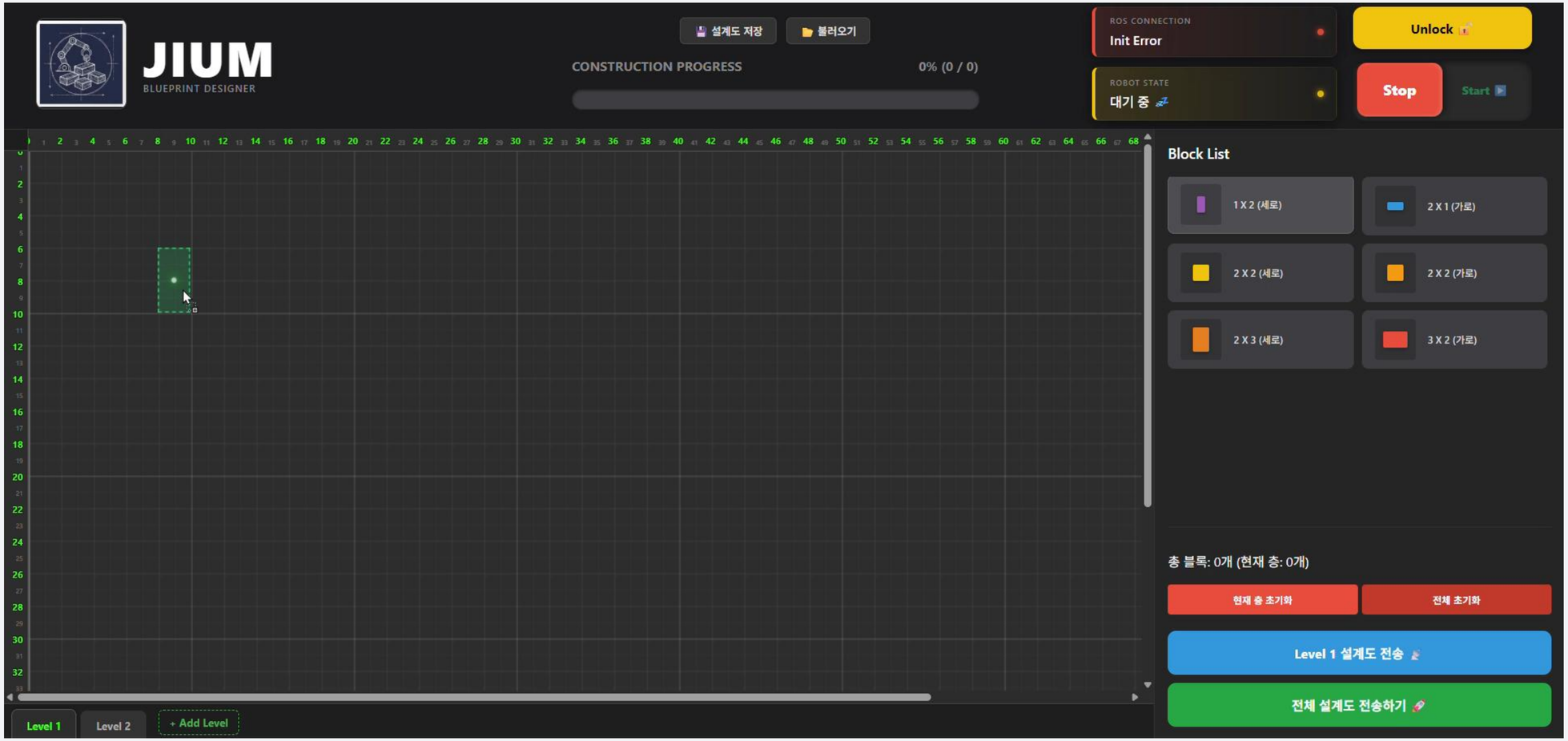
UI Manual


JIUM
BLUEPRINT DESIGNER
설계도 저장
불러오기
CONSTRUCTION PROGRESS
0% (0 / 0)
ROS CONNECTION
Connected
Unlock
ROBOT STATE
대기 중
Stop
Start
Block List
1 X 2 (세로)
2 X 1 (가로)
2 X 2 (세로)
2 X 2 (가로)
2 X 3 (세로)
3 X 2 (가로)
총 블록: 0개 (현재 층: 0개)
현재 층 초기화
전체 초기화
Level 1 설계도 전송
전체 설계도 전송하기
Level 1
+ Add Level


JIUM
BLUEPRINT DESIGNER
설계도 저장
불러오기
CONSTRUCTION PROGRESS
0% (0 / 0)
ROS CONNECTION
Connected
ROBOT STATE
대기 중
Unlock
Stop
Start
Block List
1 X 2 (세로)
2 X 1 (가로)
2 X 2 (세로)
2 X 2 (가로)
2 X 3 (세로)
3 X 2 (가로)
총 블록: 0개 (현재 층: 0개)
현재 층 초기화
전체 초기화
Level 1 설계도 전송
전체 설계도 전송하기
Level 1
+ Add Level

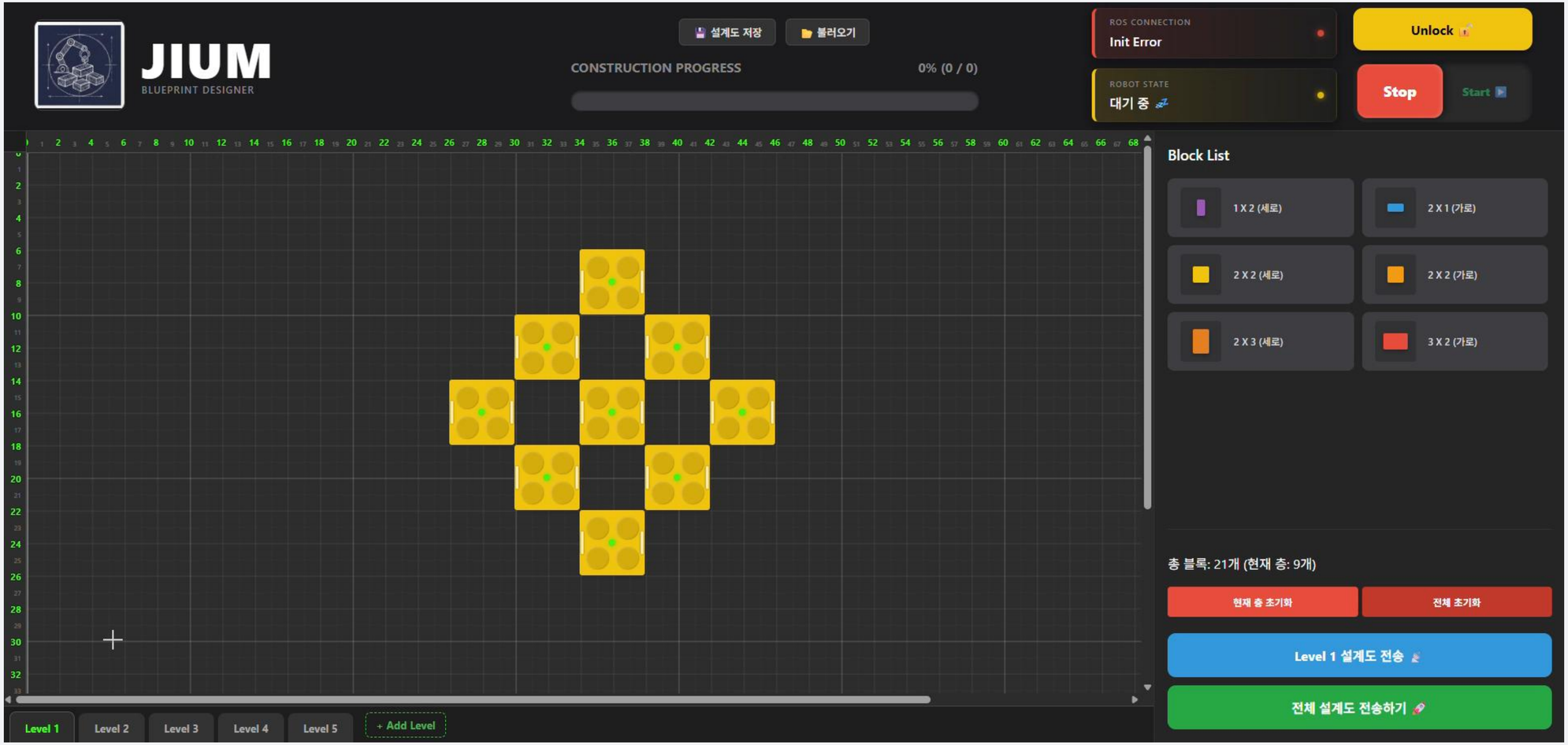
UI

place blocks (video)



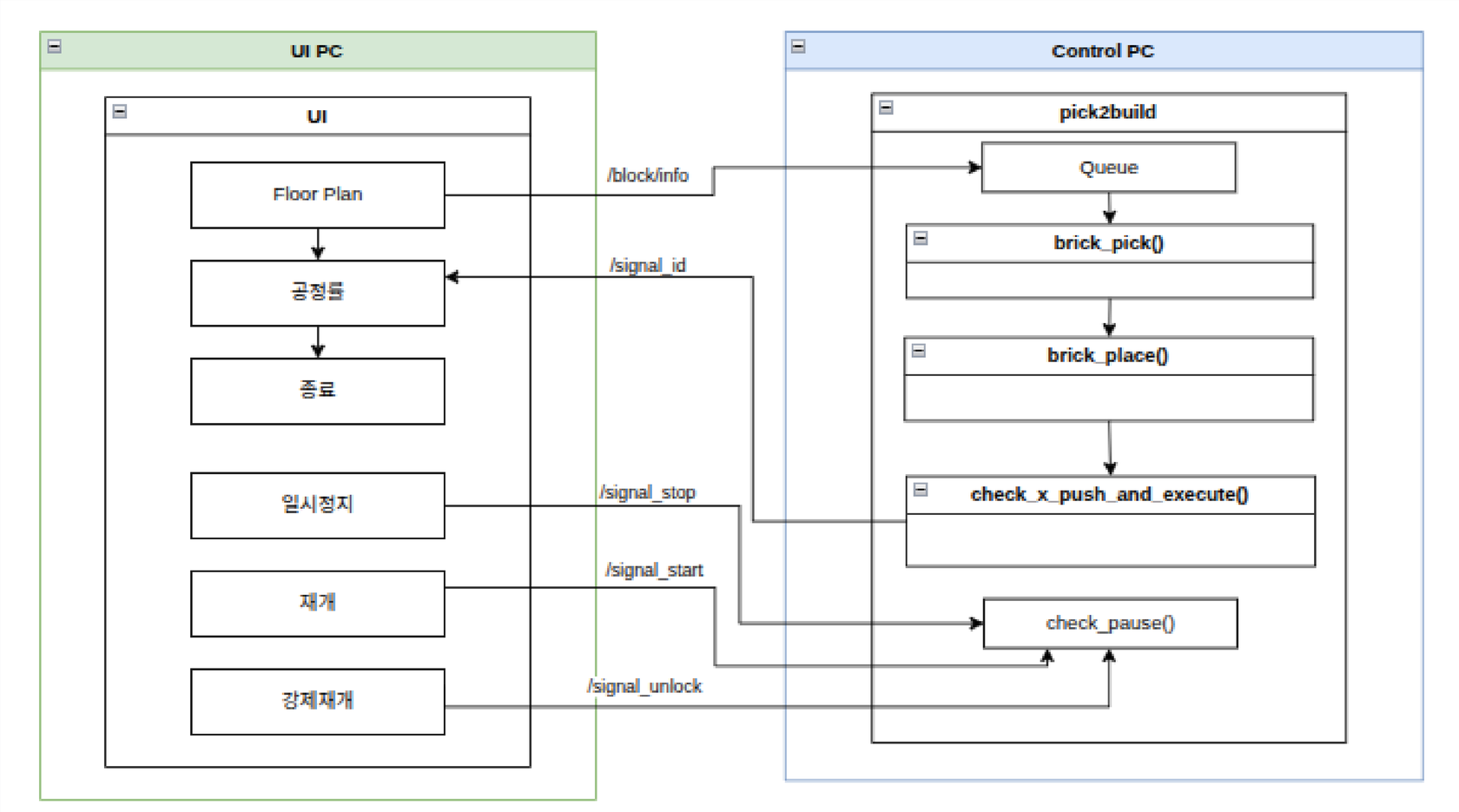
UI

save / bringup blueprint (video)



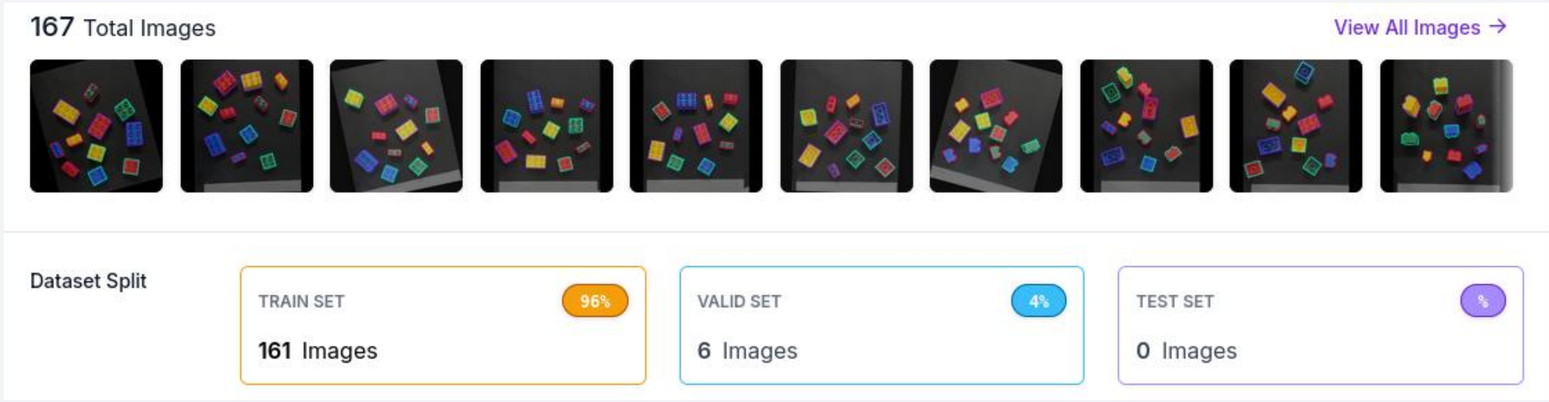
UI

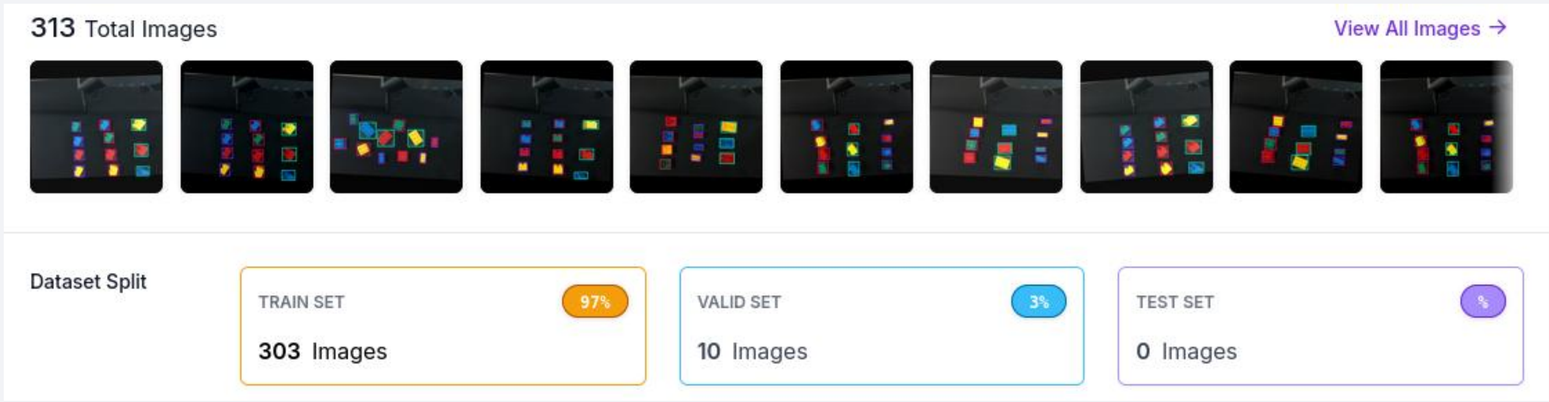
UI Node Flow



Detection - Block

YOLO Dataset (1st / 2nd)

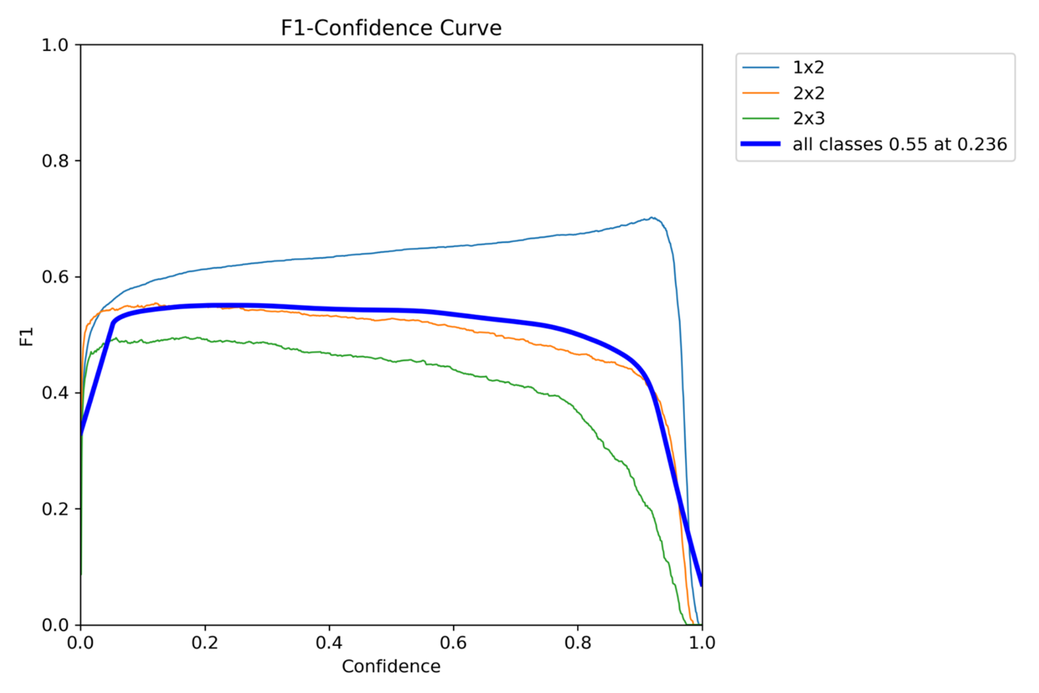


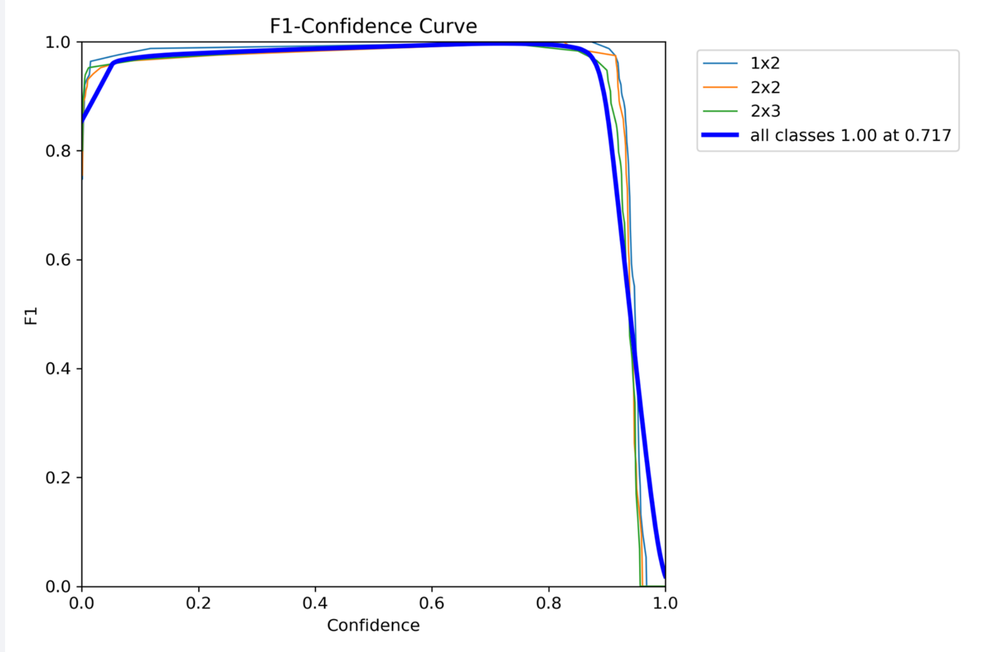


JIUM

Detection - Block

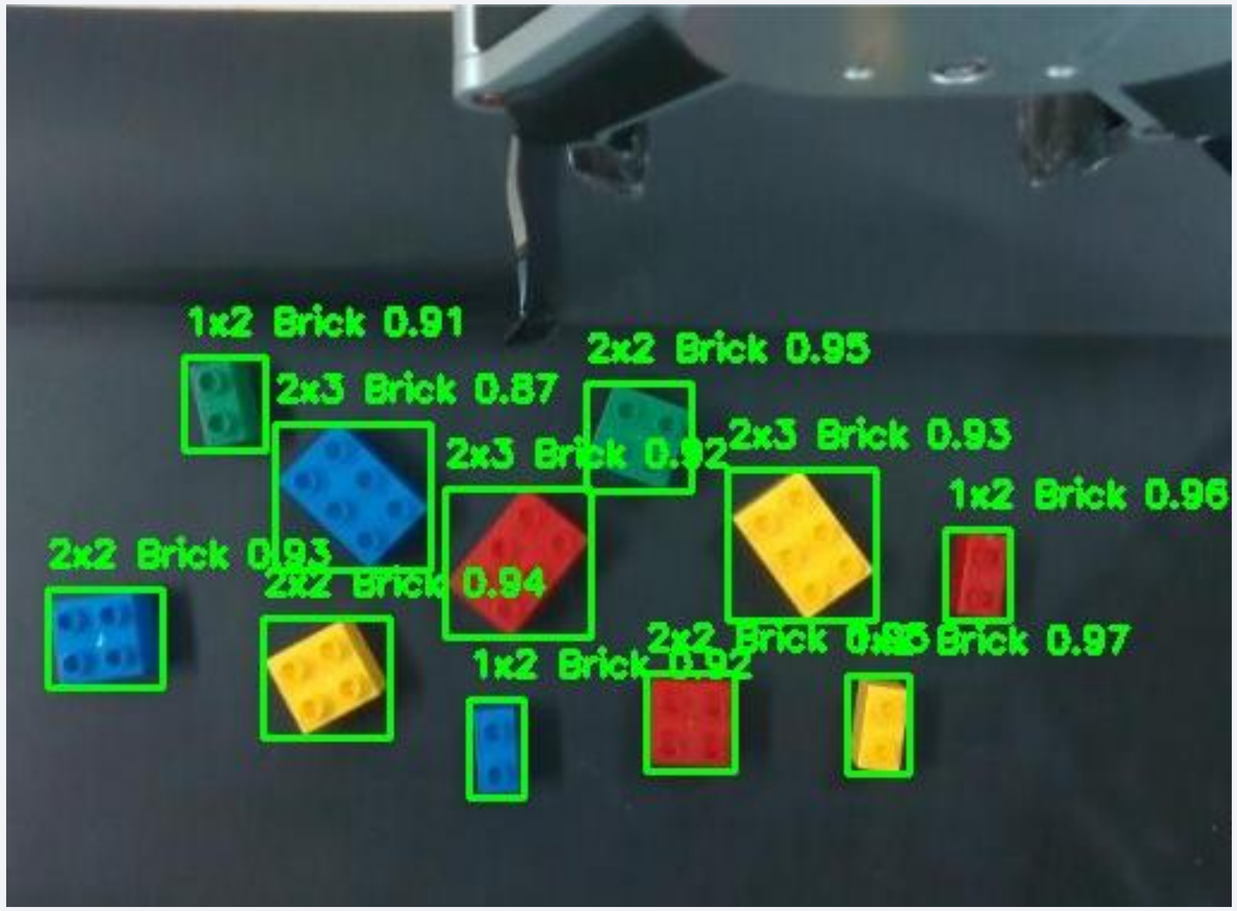
Yolo Training Result (1st / 2nd)





Detection

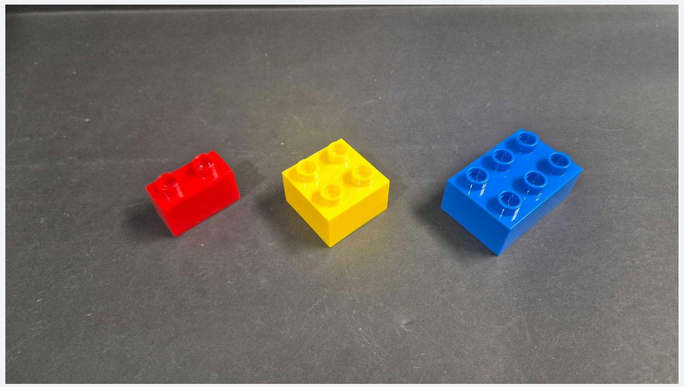
Yolo detection



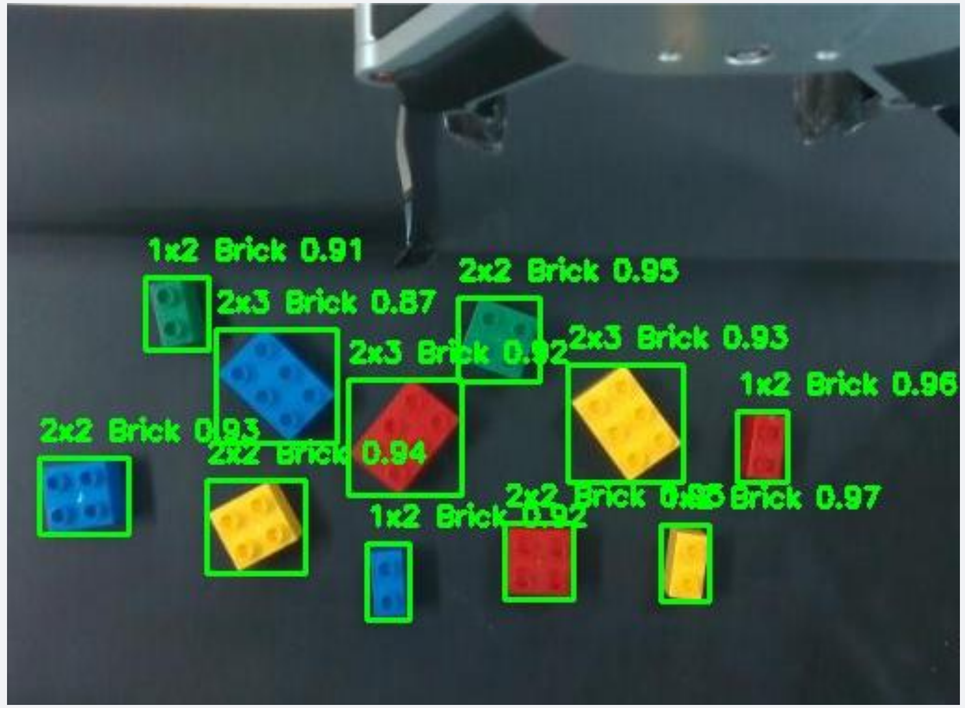
Detection

Detecting & Foundation Pose

```
/dsr01/detection_start
data: 0
---
data: 1
---
data: 2
---
```



Block ID (0 / 1 / 2)

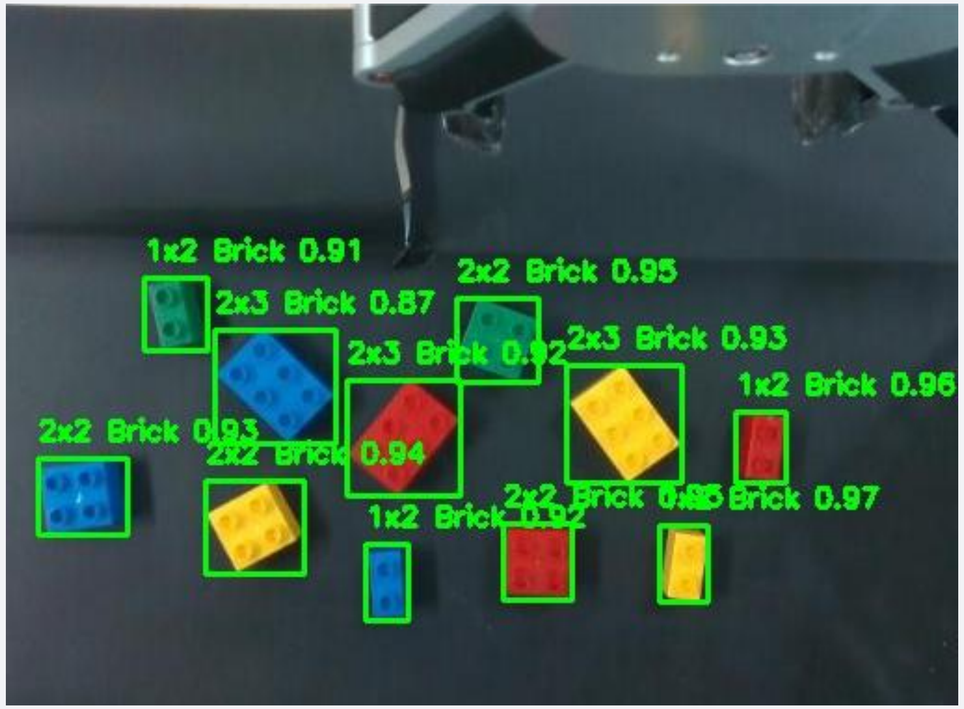


Block Detection(YOLO)

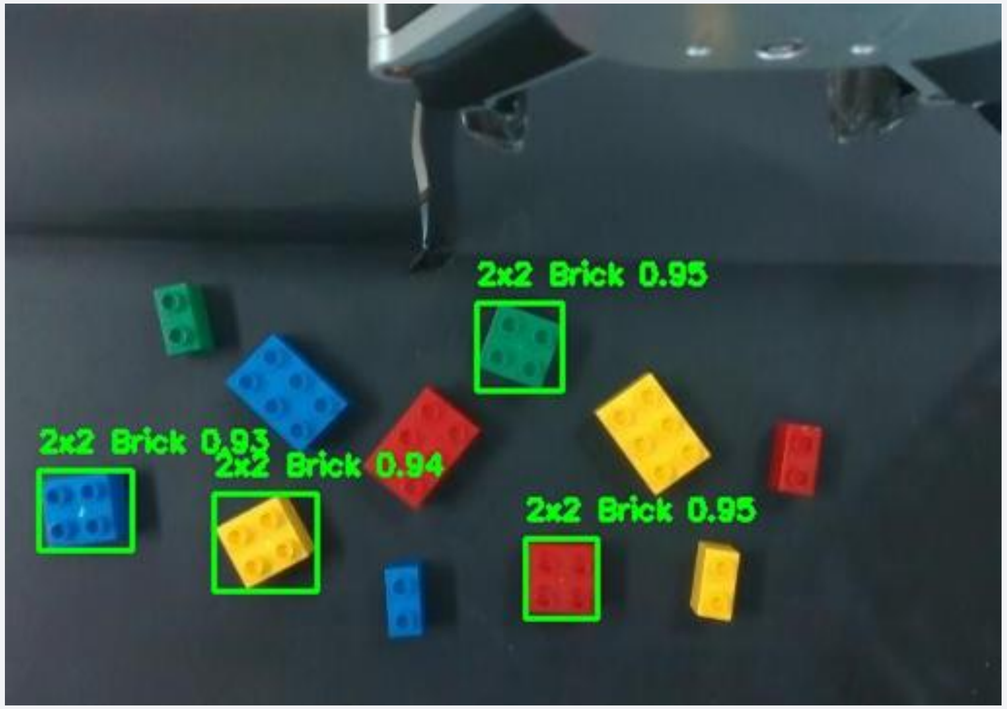
JIUM

Detection

Yolo detection



All type



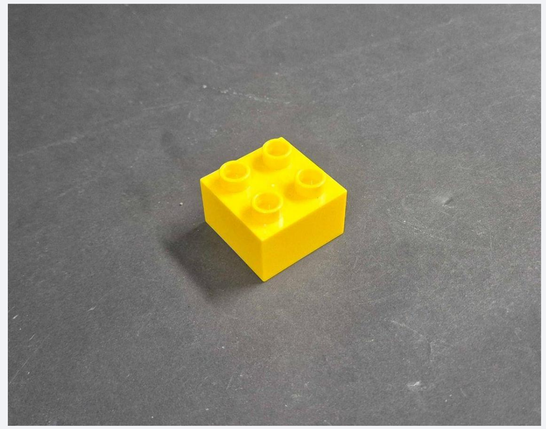
Type (Only 2X2)

Detection

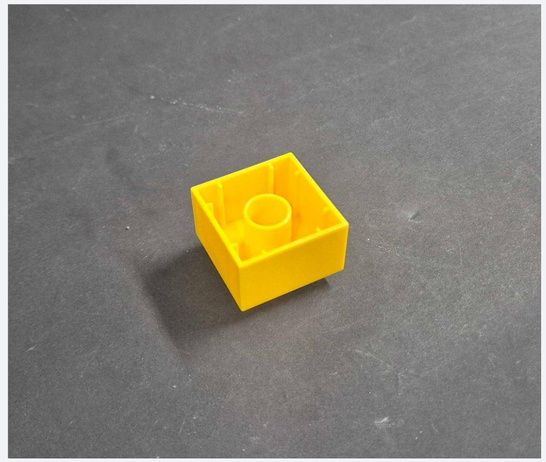
Detecting & Foundation Pose

```
---
layout:
  dim: []
  data_offset: 0
data:
- -435.69389414832676
- -338.0400226540336
- -13.998537790755783
- -4.230673564671175
- -5.618897590403699
- 74.28642026637337
- 0.0
---
```

Get Block Position & Pose



```
ID 1 | Pose Code: 0.0 | 상태: 재정렬 불필요
```



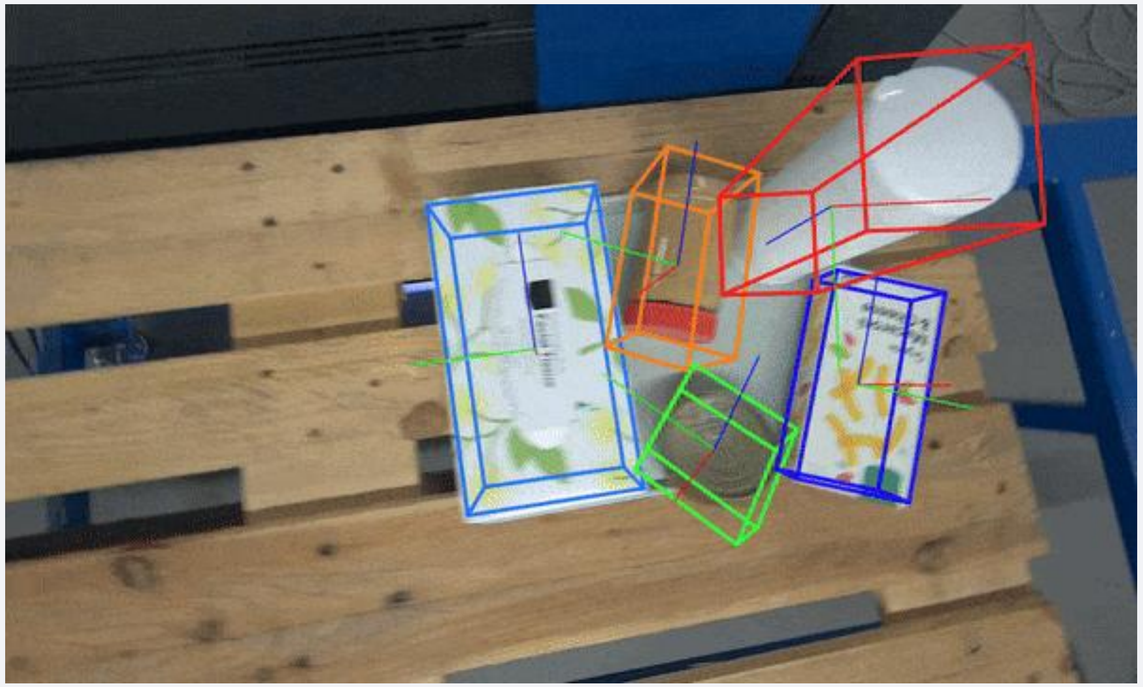
```
ID 1 | Pose Code: 1.0 | 상태: 재정렬 필요
```

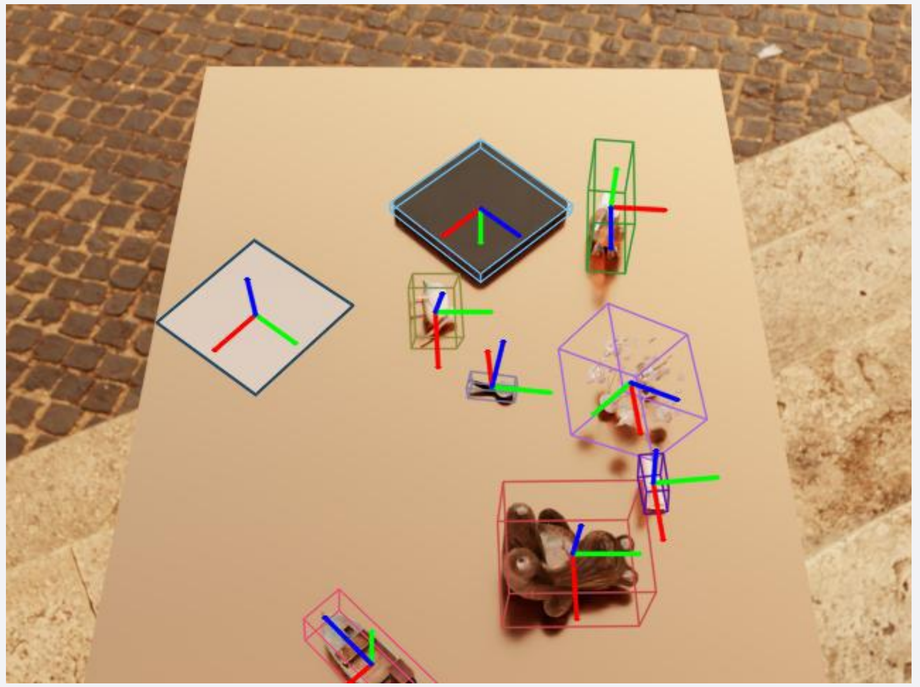
Publish Block Position & Pose

Detection - Block

Foundation Pose

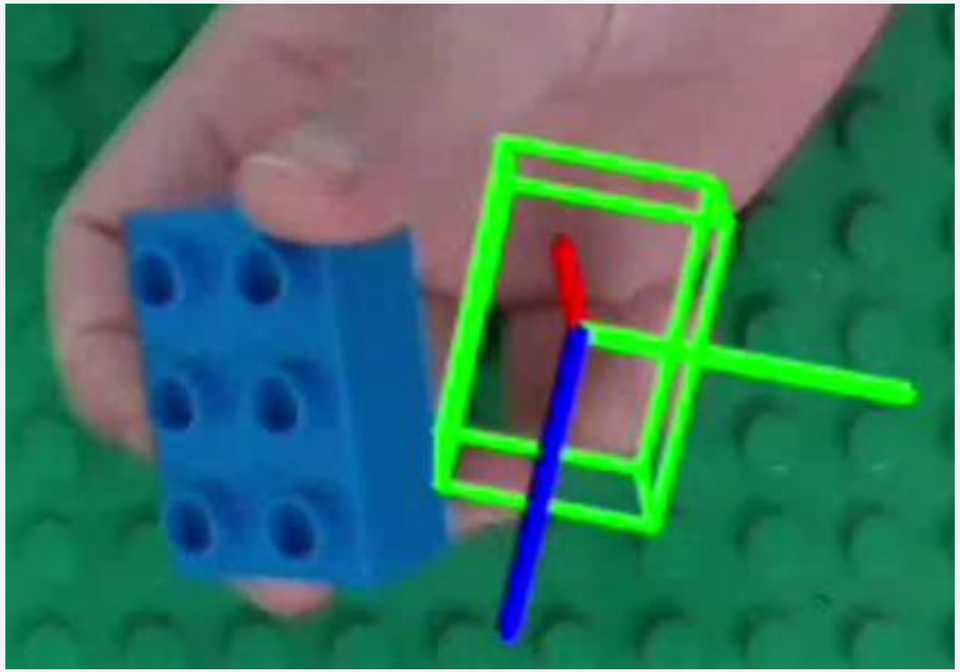
NVIDIA FoundationPose

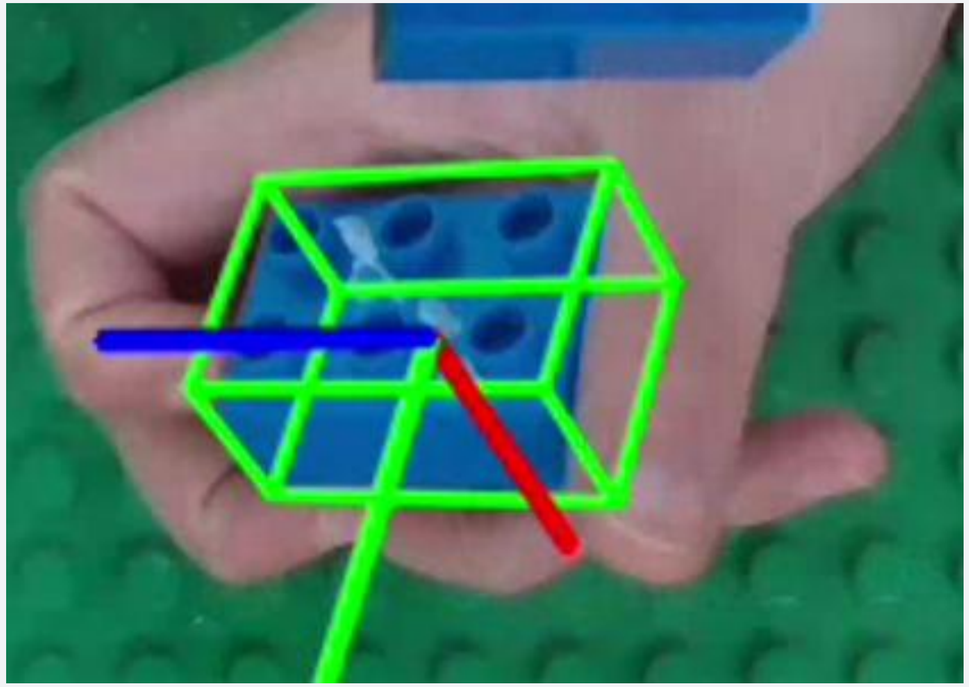




Detection - Block

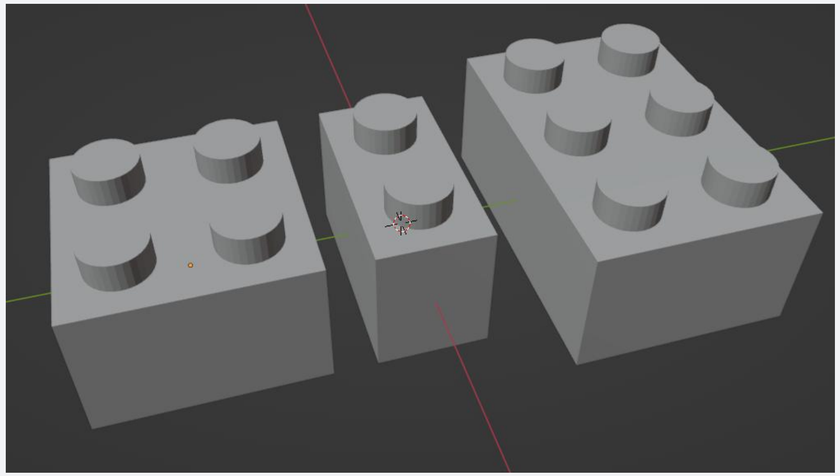
Foundation Pose

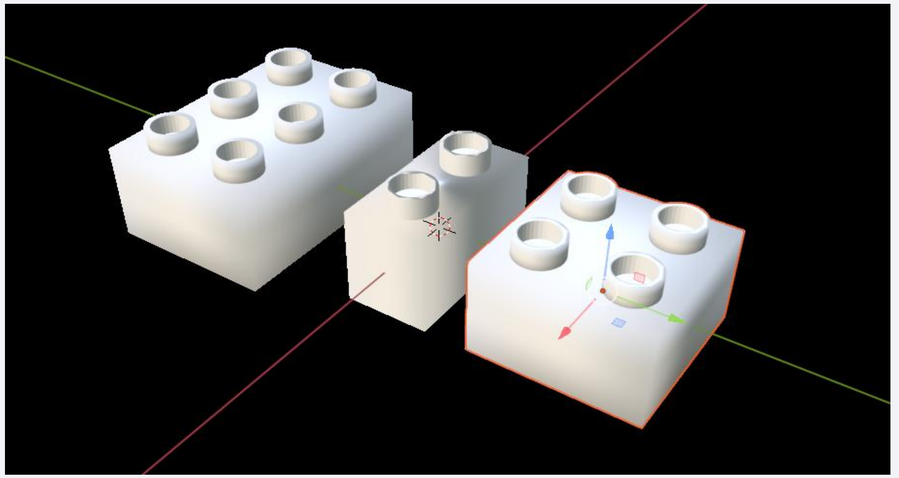


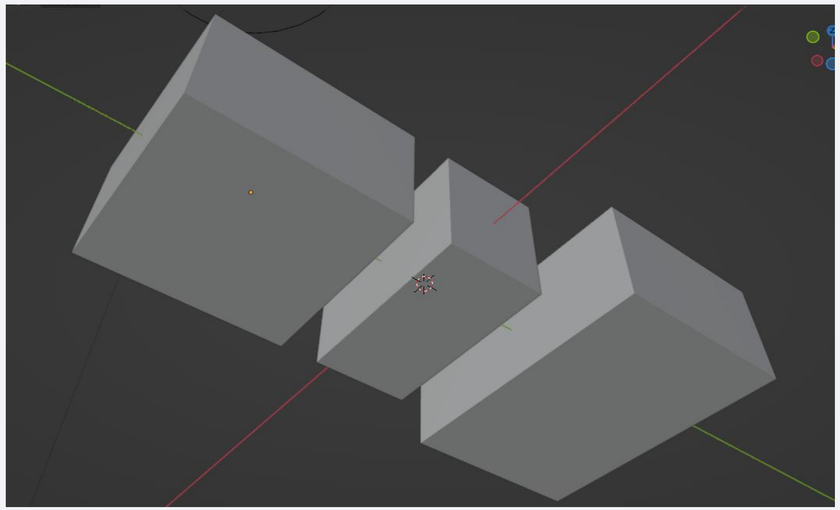


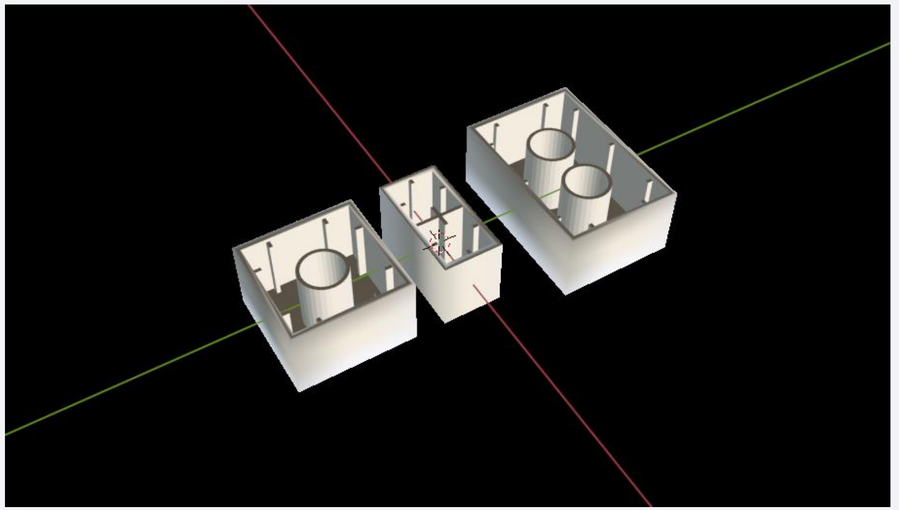
Detection - Block

3D modeling



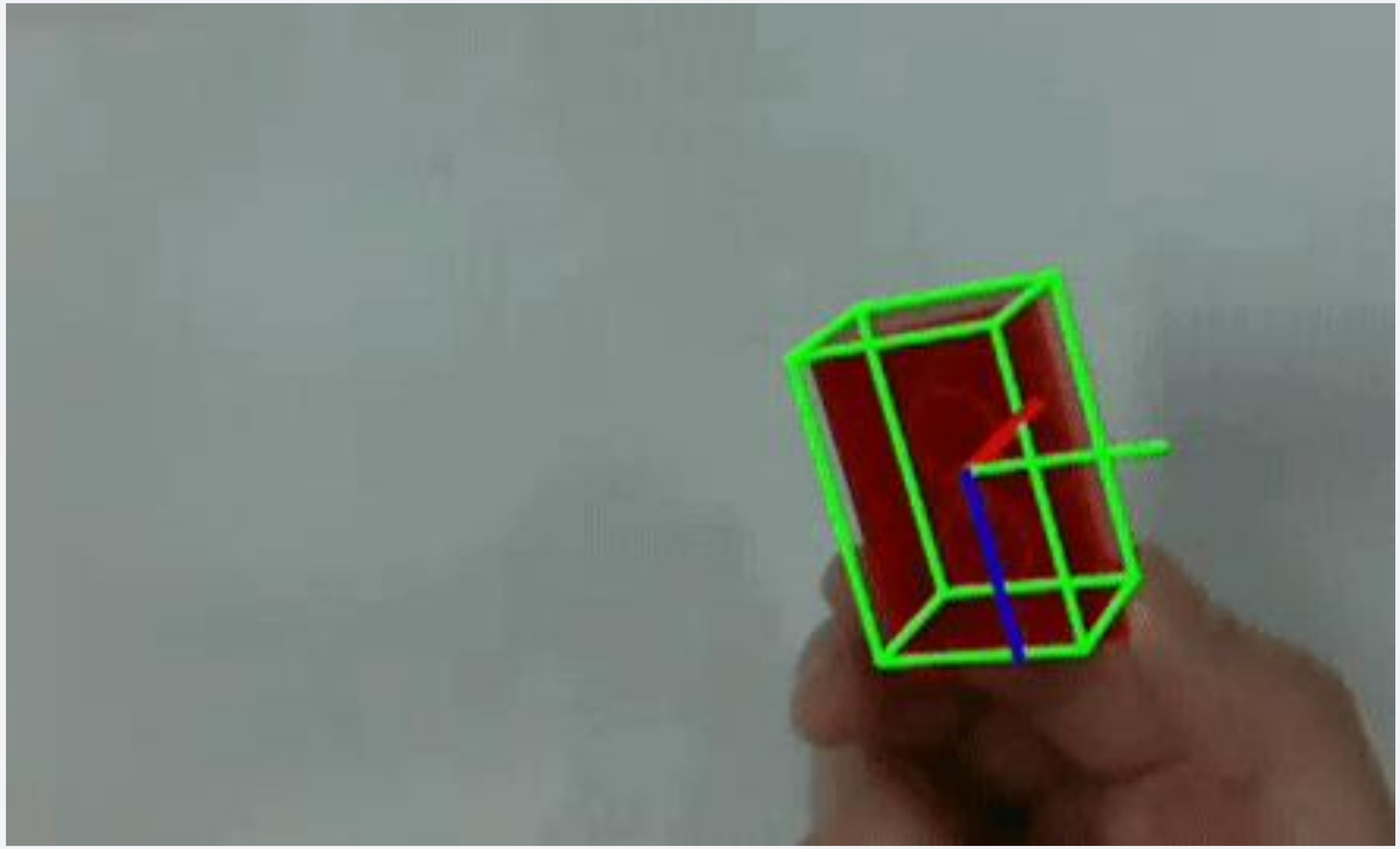


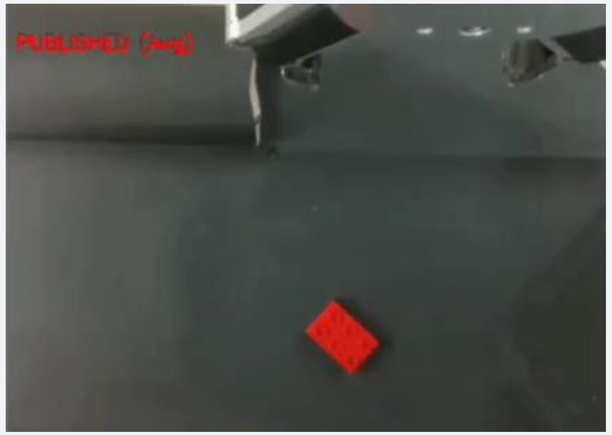




Detection

Foundation Pose (+ 3d modeling)

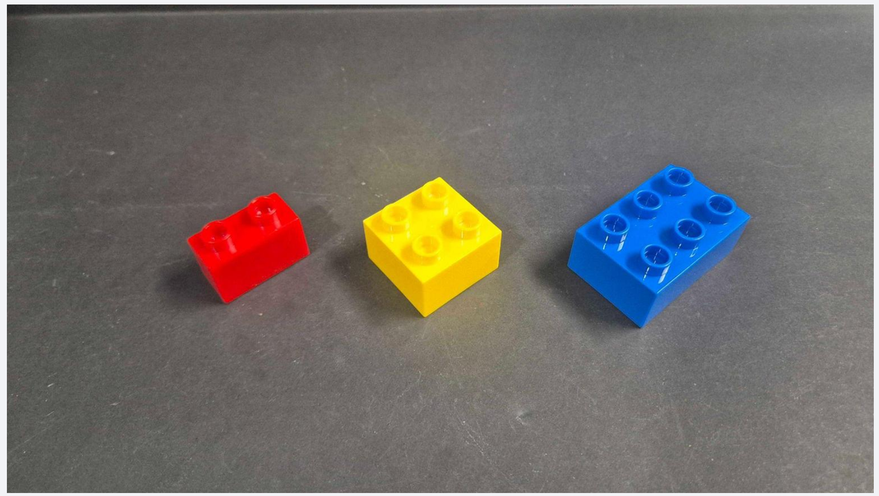




JIUM

Detection

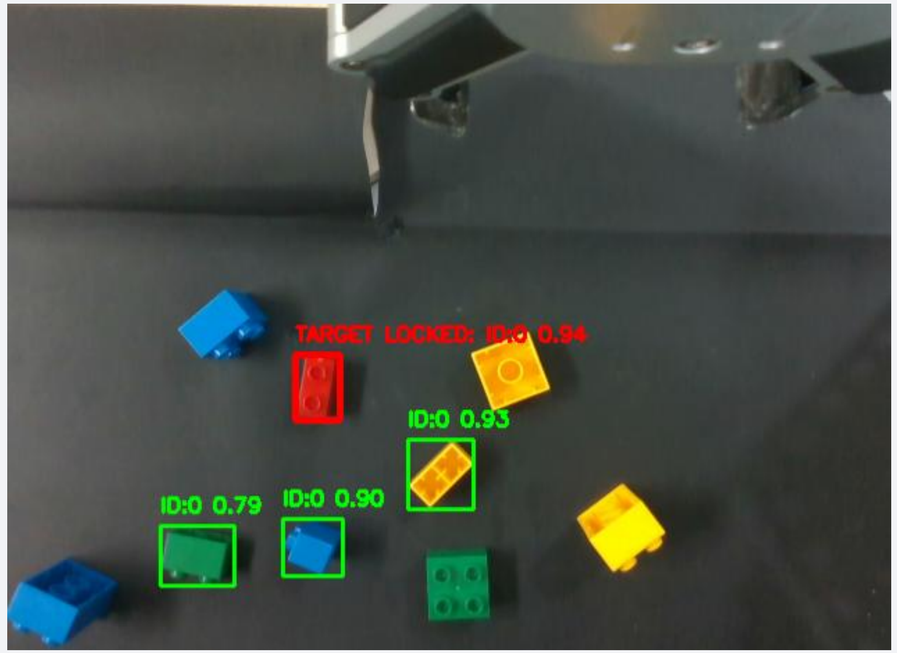
Detecting & Foundation Pose



```
/dsr01/detection_start
data: 0
---
```

Subscribe Block ID

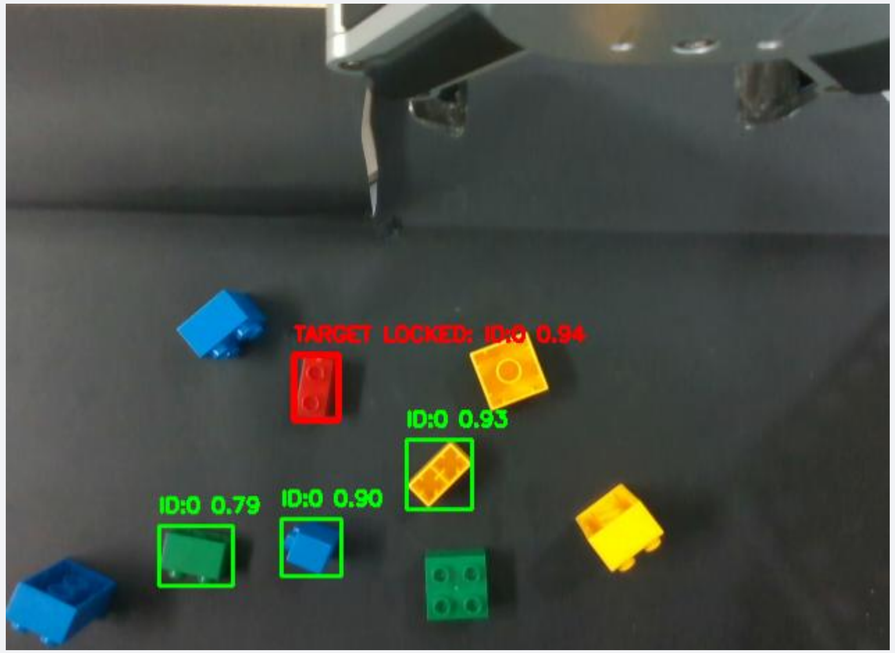
(0/ 1 / 2)

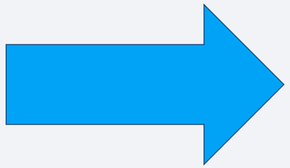


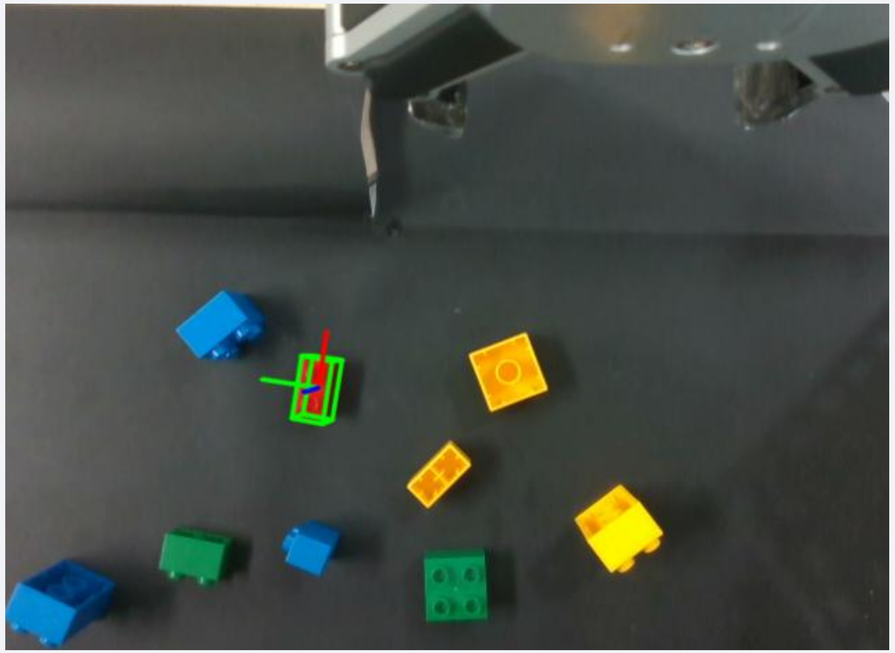
Block Detection(YOLO)

Detection

block detection + pose estimation







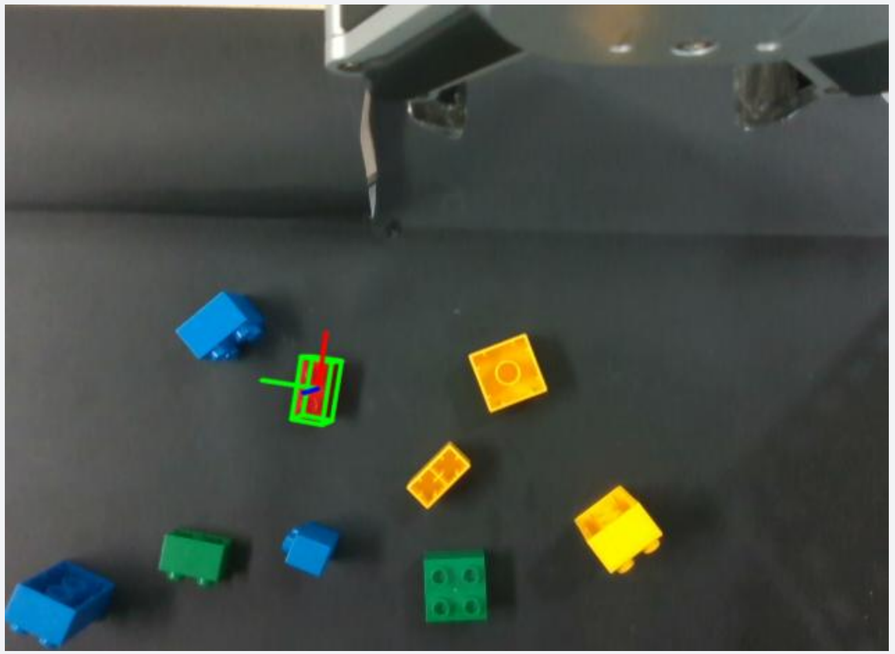
Block Detection(YOLO)

Pose Estimation

(Foundation Pose)

Detection

Detecting & Foundation Pose



Pose Estimation

(Foundation Pose)

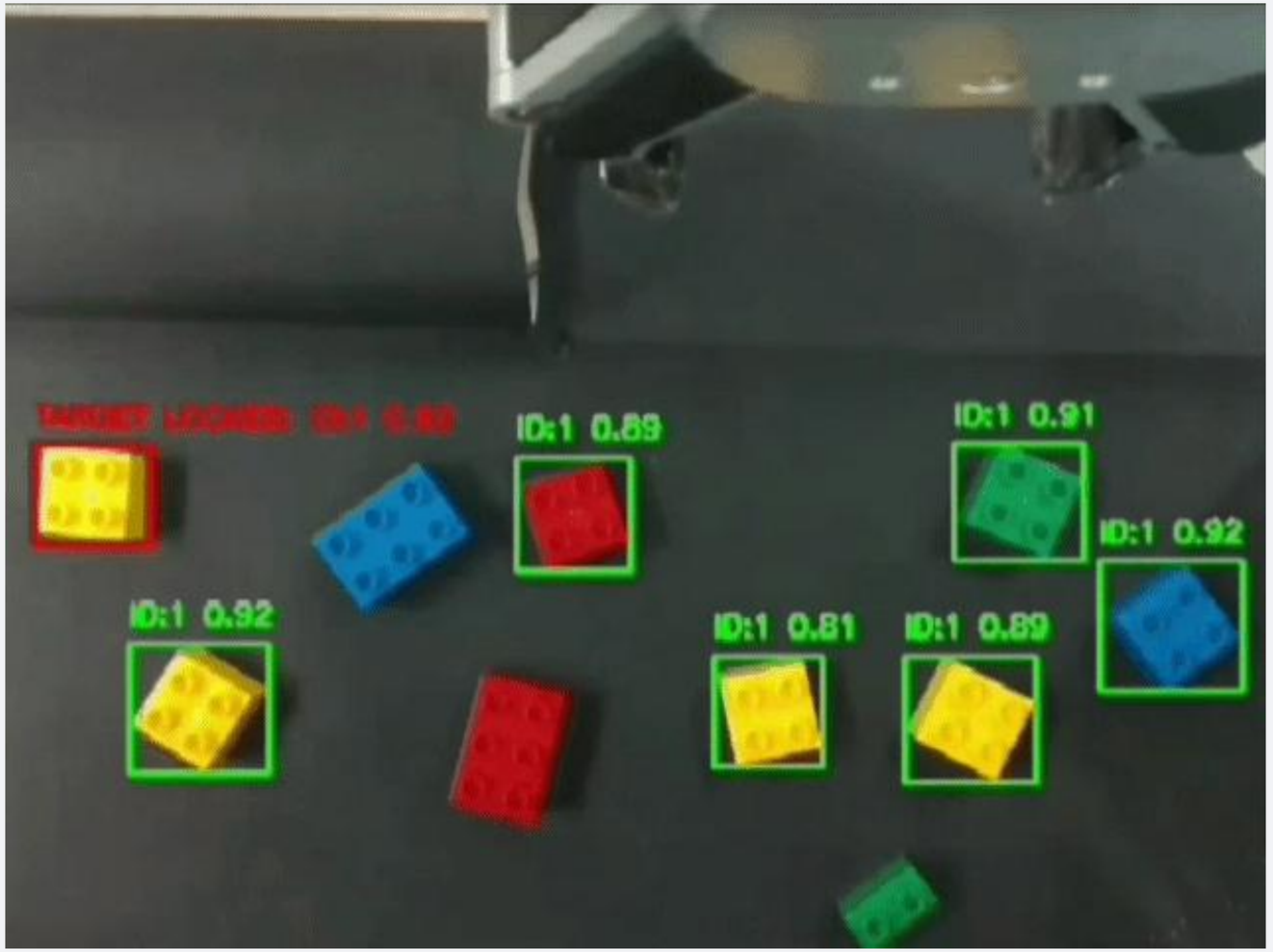
```
layout:
  dim: []
  data_offset: 0
data:
- -435.69389414832676
- -338.0400226540336
- -13.998537790755783
- -4.230673564671175
- -5.618897590403699
- 74.28642026637337
- 0.0
```

6 DOF (x, y, z, a, b, c)

+ Pose

Detection

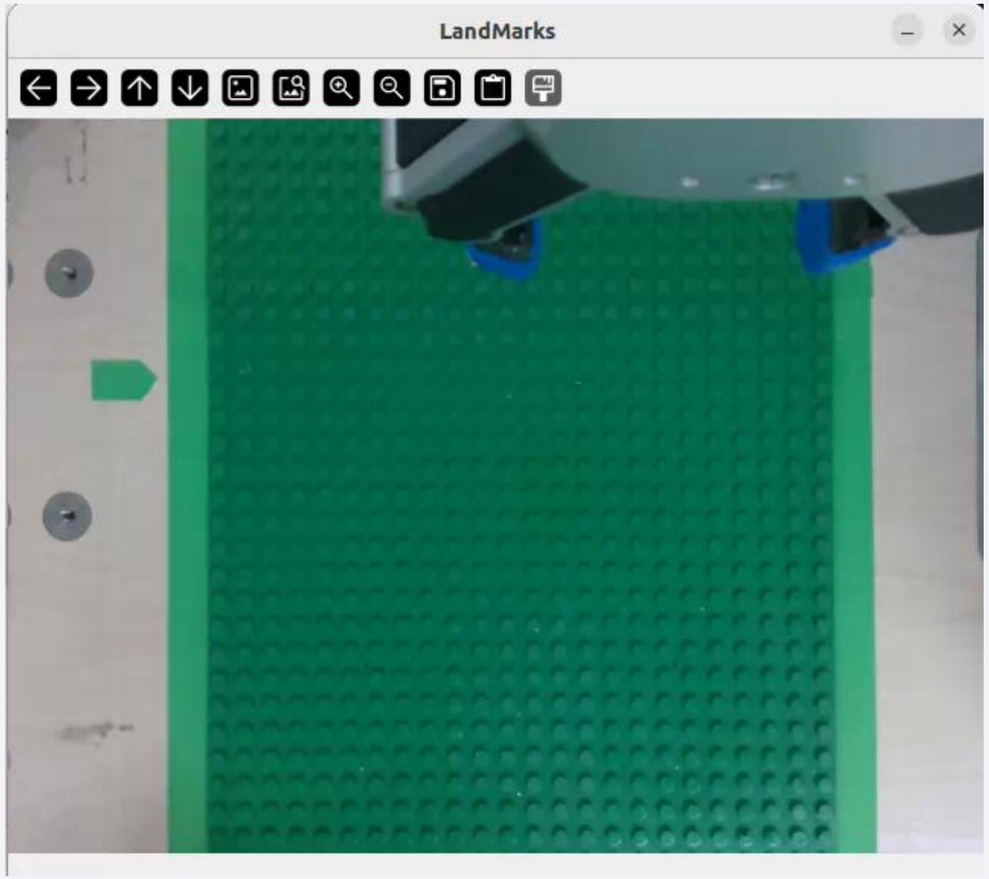
block detection + pose estimation (video)

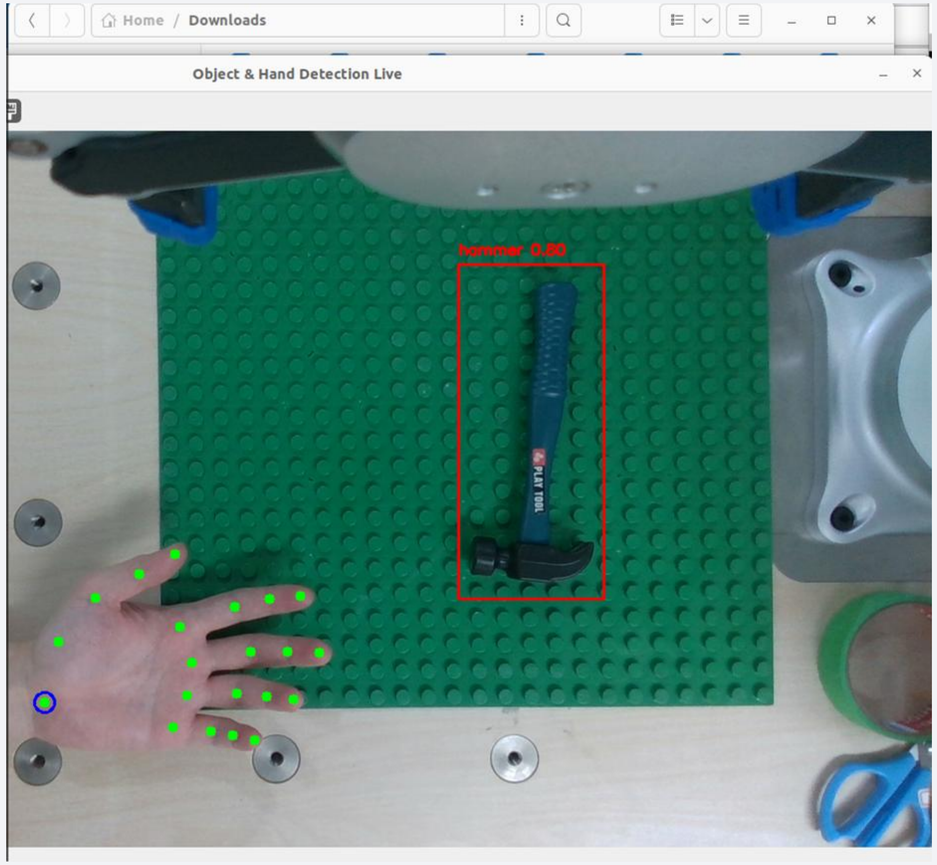


Detection - Hand & Equipment

MediaPipe

hand & equipment detection (video)

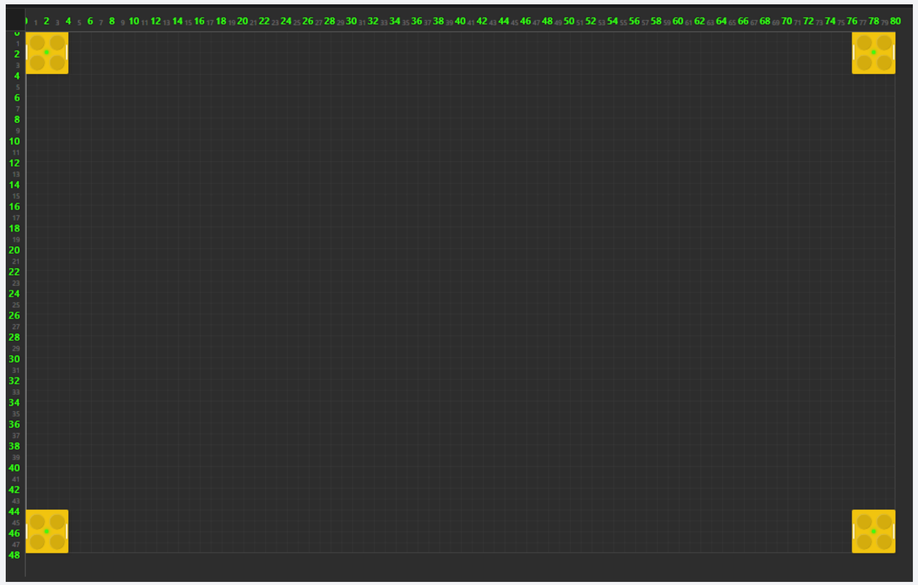




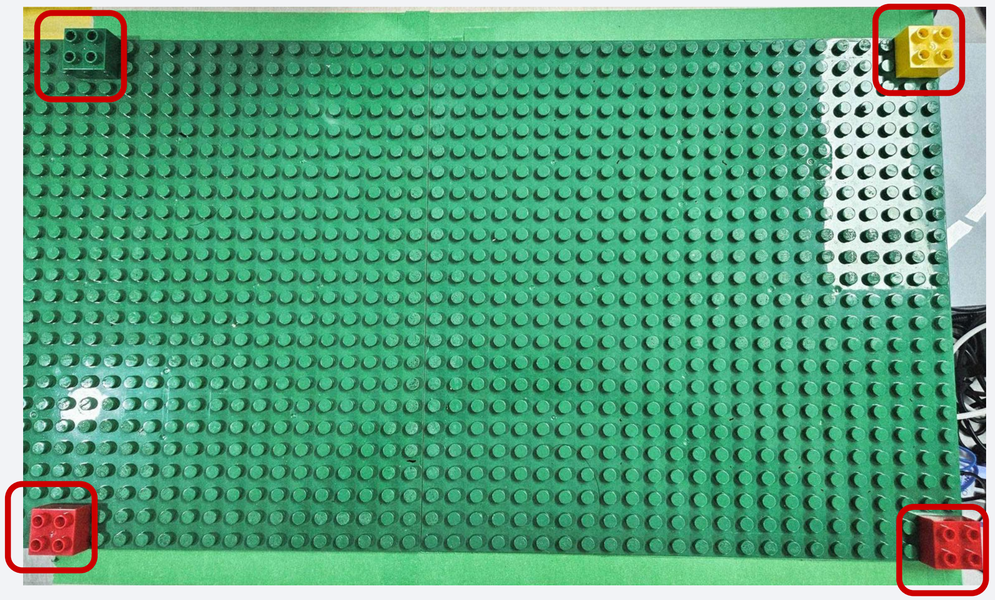
JIUM

Control

Coordinate Mapping



U I



Real world

변환행렬 사용

레고판의 좌상단, 우상단, 좌하단, 우하단의 좌표 매핑

변환행렬을 사용하여 전체 레고판 좌표 변환

Control

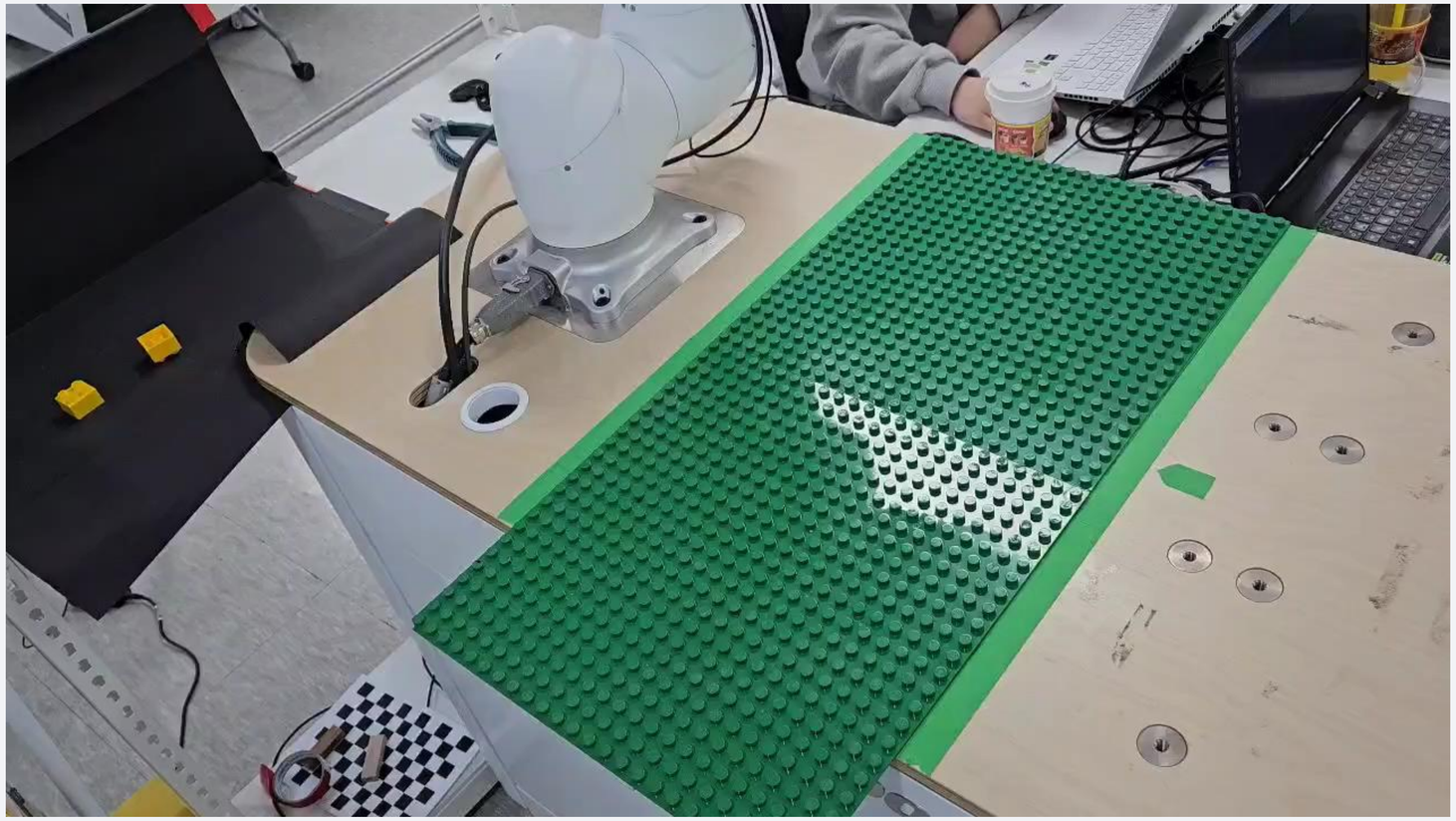
Coordinate Mapping

```python
def get_transformation_matrix():
    src_pts = np.array([[2, 2],[80, 2],[80, 46],[2, 46]], dtype=np.float32)
    dst_pts = np.array([[571.52,309.13],[572.65,-313.56],[221.37,-313.85],[220.93,309.72]],dtype=np.float32)
    H, _ = cv2.findHomography(src_pts, dst_pts)
    return H

def grid_to_base(u, v, H):
    point = np.array([[[u, v]]], dtype=np.float32)
    t_point = cv2.perspectiveTransform(point, H)
    return t_point[0][0][0], t_point[0][0][1]
```

Control

stage 1 : brick pick



Stage 1 flow

1. UI에서 /block/info 수신
2. brick_pick area 이동
3. /dsr01/detection_start 수신
4. detection pc에서 brick 좌표와 벡터 전송
5. brick_pick
6. rearrange_area 이동

7.1 brick이 정면이 아닌 경우 손을 인식하여 손으로 전달

7.2 brick이 정면인 경우 재정렬 수행

Control

stage 2 : brick place



Stage 2 flow

1. brick_pick

2. brick_place area 이동

3.1 그리퍼 진입이 불가한 경우 손을 인식하여 손으로 전달

3.2 그리퍼 진입이 가능한 경우 목표 좌표에 brick place

Control

stage 2 : brick place

순응제어와 힘제어를 활용한 brick place

```
if block_type == "1" or block_type == "2":
    self.force = 20
    if block_type == "1":
        self.period = [1.0, 0.3, 0, 0, 0, 1.0]
    if block_type == "2":
        self.period = [0.3, 1.0, 0, 0, 0, 1.0]

elif block_type == "3" or block_type == "4":
    self.force = 19
    self.period = [0.5, 0.5, 0, 0, 0, 1.0]

elif block_type == "5" or block_type == "6":
    self.force = 22
    self.period = [1.0, 1.0, 0, 0, 0, 1.0]
```

Compliance & Force

순응 제어

강성 : [3000,3000,100,200,200,200]

힘 제어

brick 별 다른 force 적용

Move periodic

주기 운동

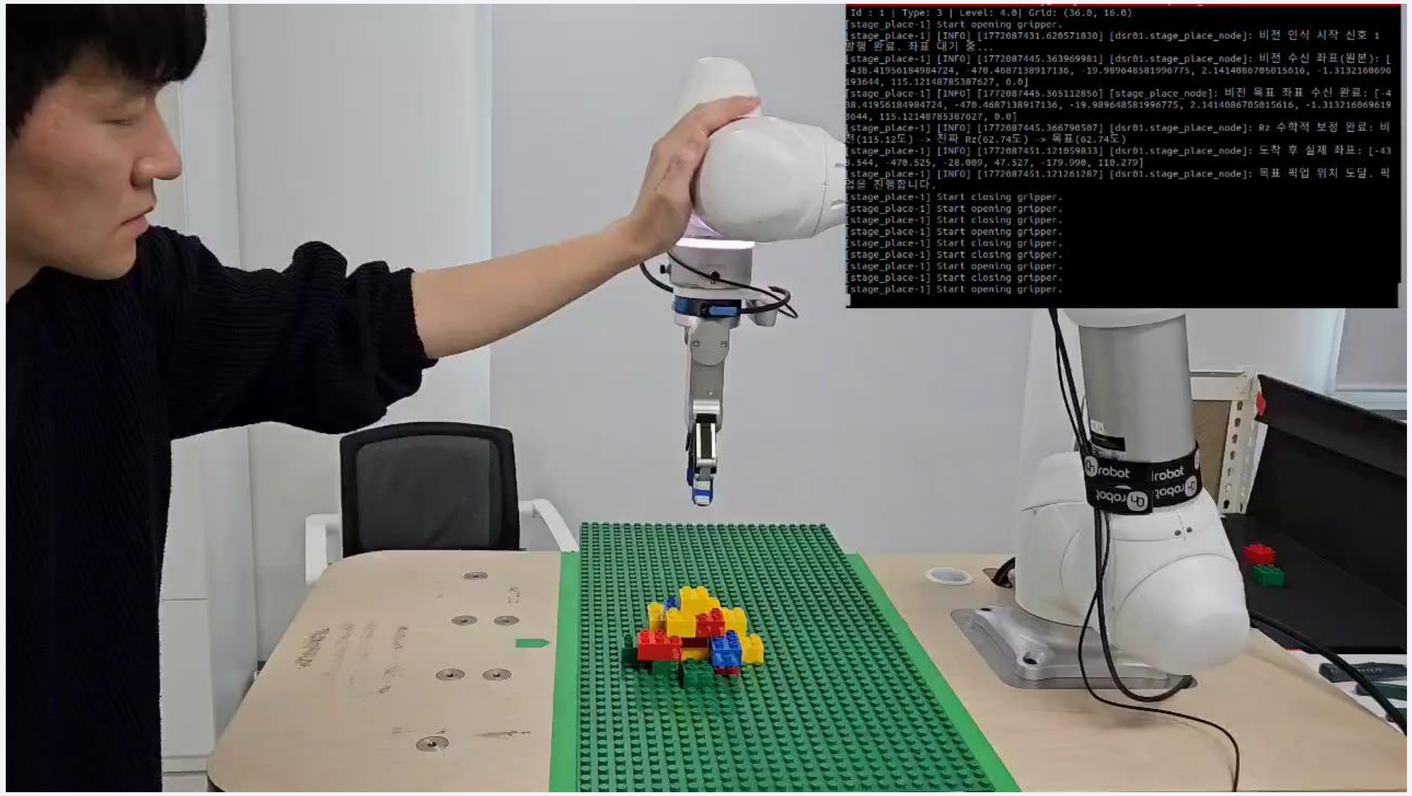
brick 별 다른 주기 운동 활용

주기운동 최적화

JIUM

Control

stage 3 : User Push



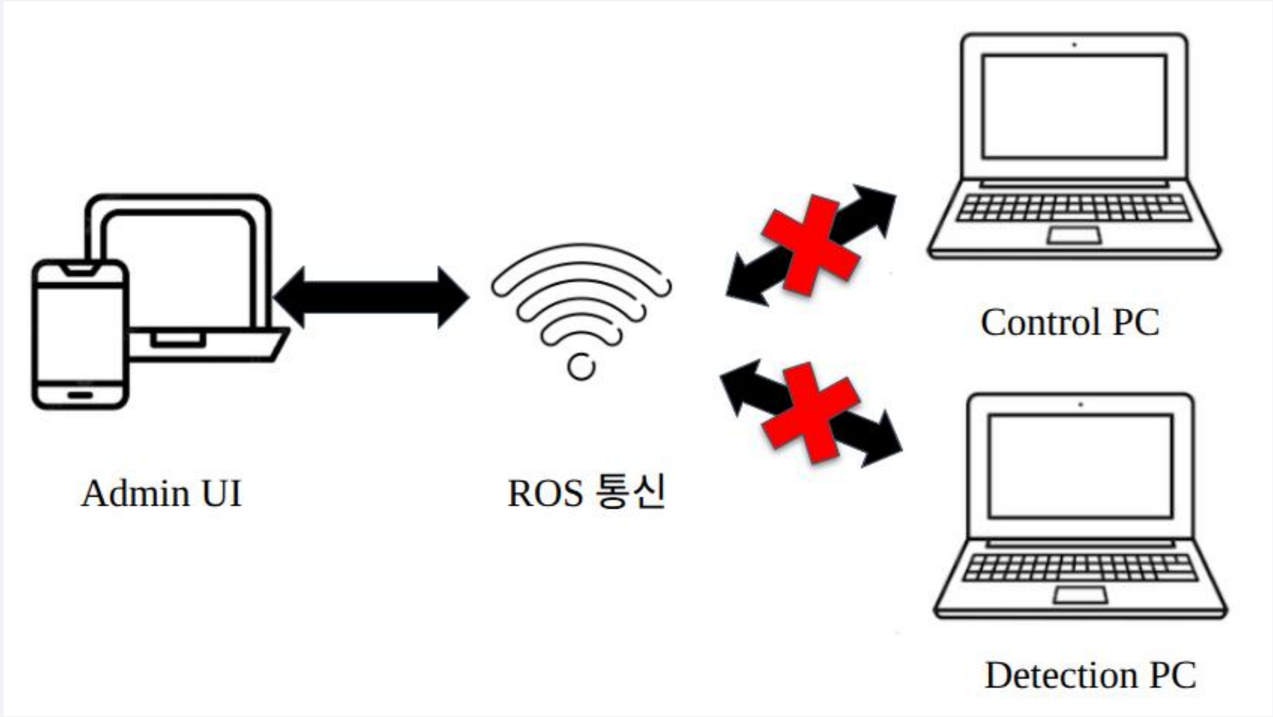
Stage 3 flow

1. 목표 좌표 상단 위치에서 5초간 대기
2. 대기시간동안 힘을 감지한 경우 보이스 인식 시작
3. 보이스에서 목표 위치와 목표를 추출
4. tool area로 이동하여 목표 탐색
5. tool pick 후 목표 위치로 전달

Control

ROS2 & Topic 통신 구현

문제 발생 : 서로 다른 ROS2 버전 간 토픽 통신 불가

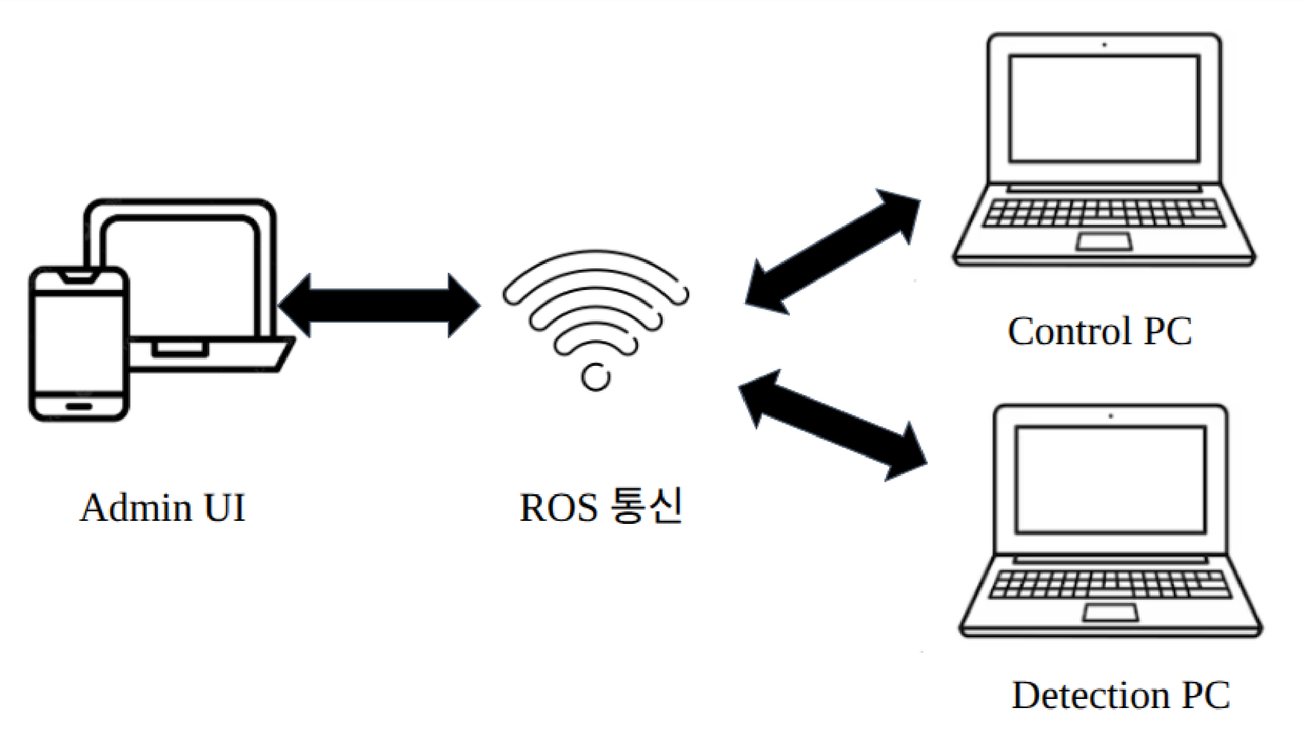


문제 상황

ROS2 Foxy와 ROS2 Humble 간의 통신에서 Fast DDS 통신 사용 시 토픽 통신이 불가한 문제 발생

Control

ROS2 & Topic 통신 구현



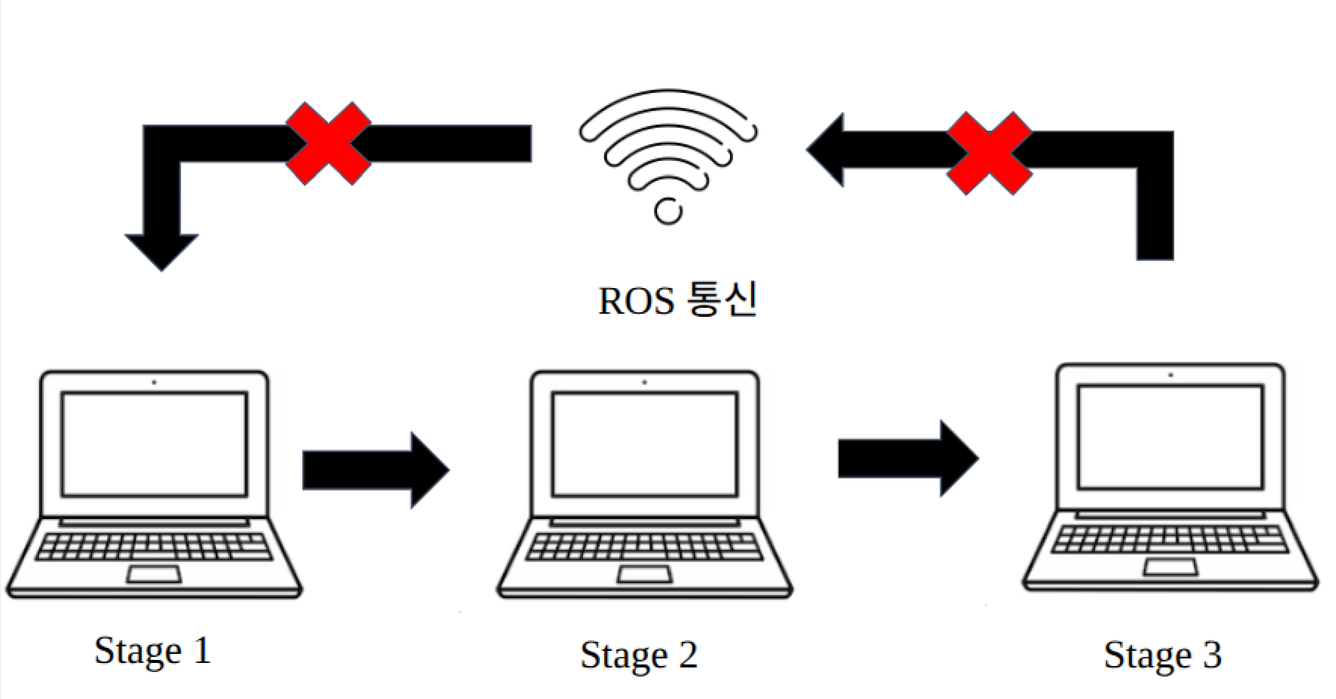
문제 해결

1. Cyclone DDS 통신 설정
2. publisher/subscriber Qos 일치 확인
3. ROS_DOMAIN_ID 일치 확인
4. PC 방화벽 해제
5. 모든 토픽은 전역 토픽으로 지정

Control

완성형 루프 로직 구현

문제 발생 : 공정 1회 실행 후 다시 재개되지 않는 문제 발생

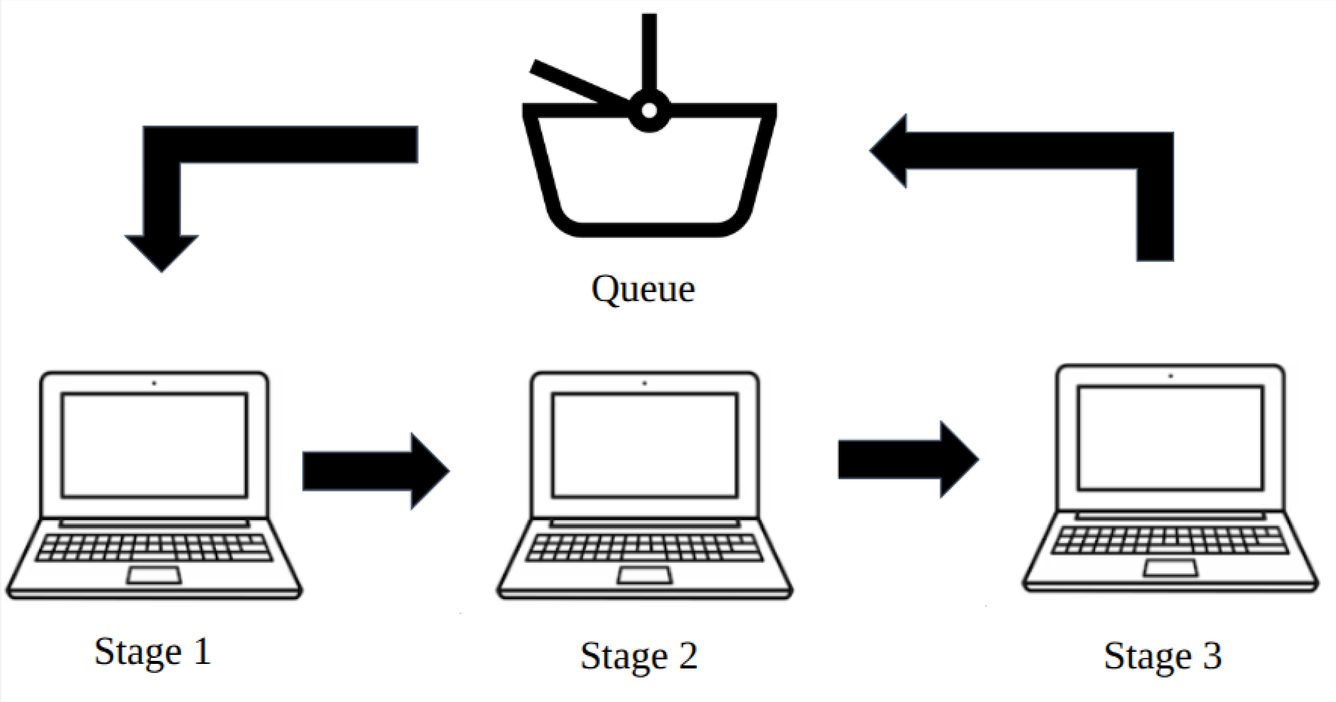


문제 상황

토픽 구독 기능과 로봇 제어 기능을 한 노드에 작성하여 Singlethread Executor으로 동작 시켰을 때 공정 수행 과정과 공정 수행 후 토픽을 받지 못하는 현상이 발생

Control

완성형 루프 로직 구현



문제 해결

Queue 구조 사용

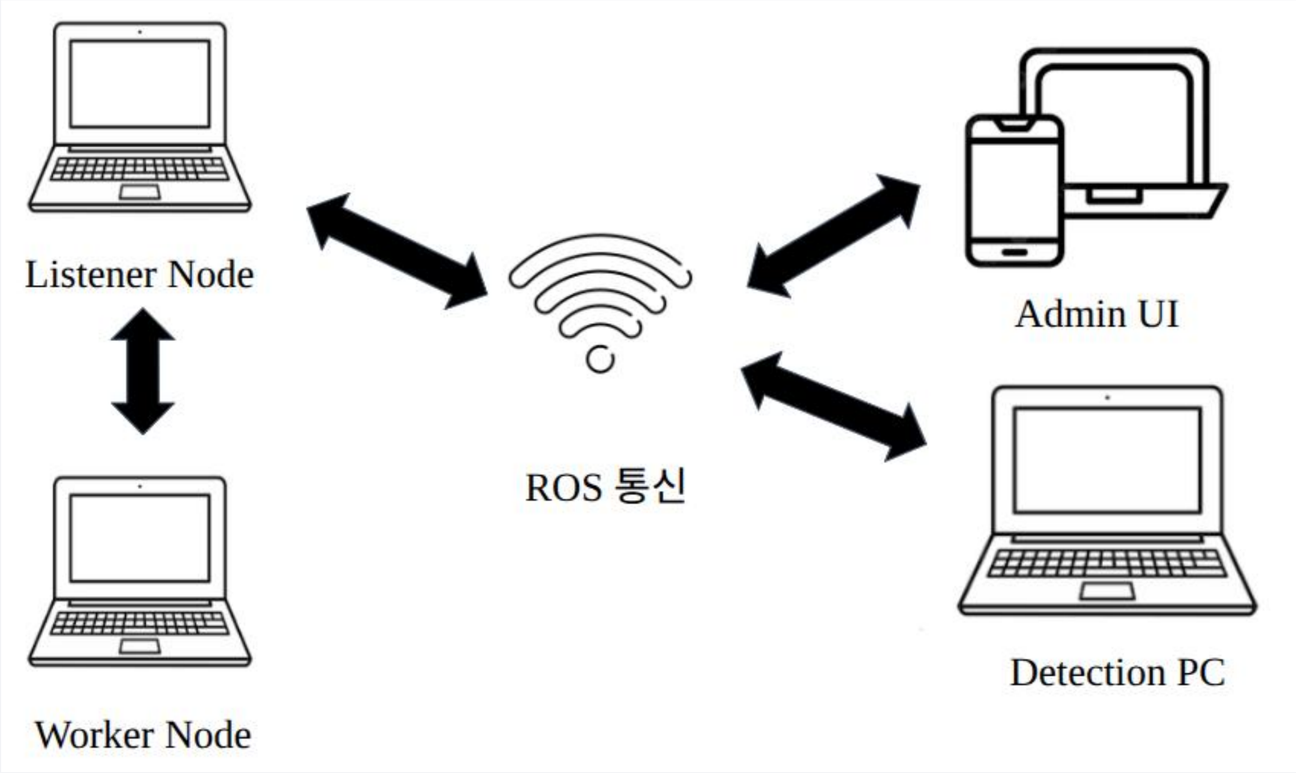
/brick/info 수신 시

Queue에 저장하고 매 루프마다 꺼내서 사용

Queue에 저장물이 없는 경우 무한 루프

Control

완성형 루프 로직 구현



문제 해결

Multithread Executor 사용

노드 분리

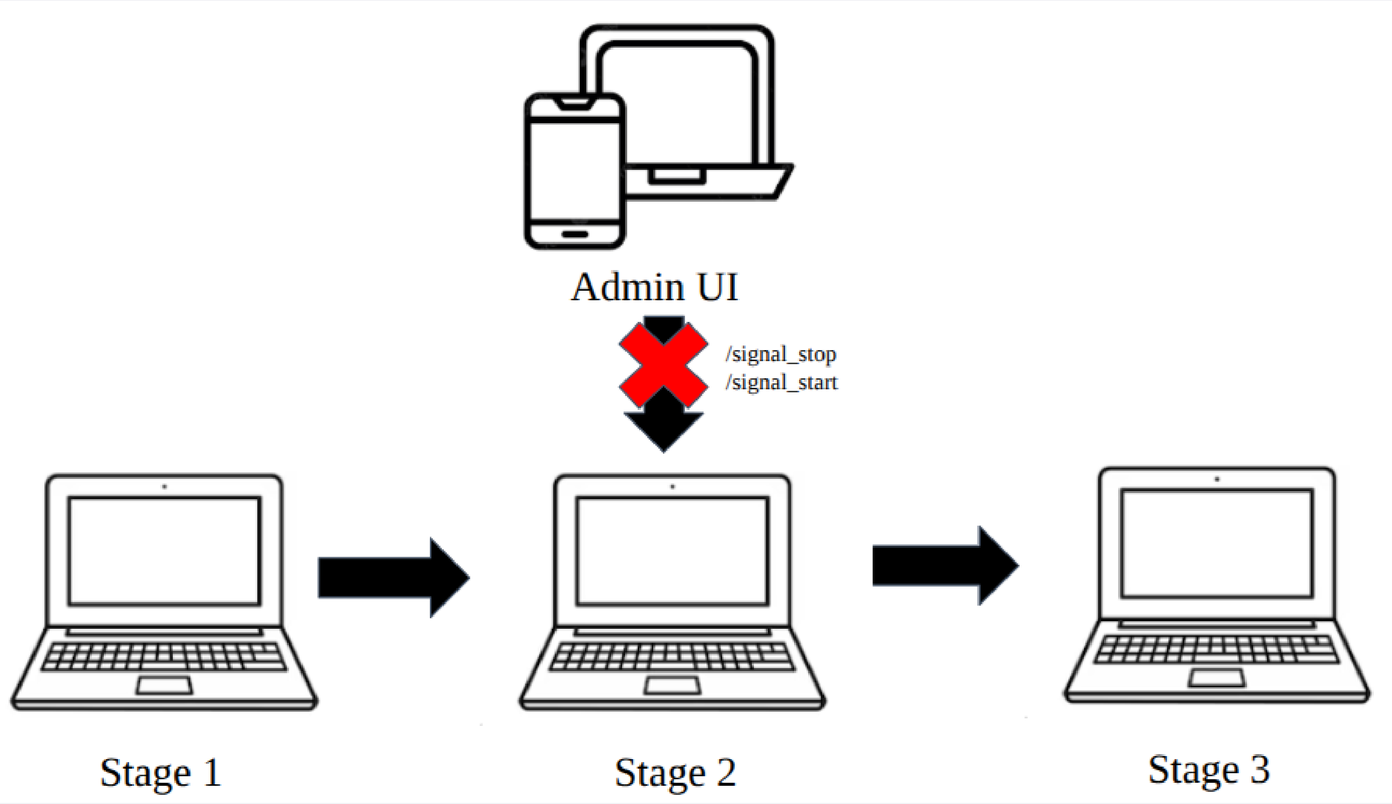
Topic Listener Node : 토픽 송수신 역할

Robot Worker Node : 로봇 제어 역할

Control

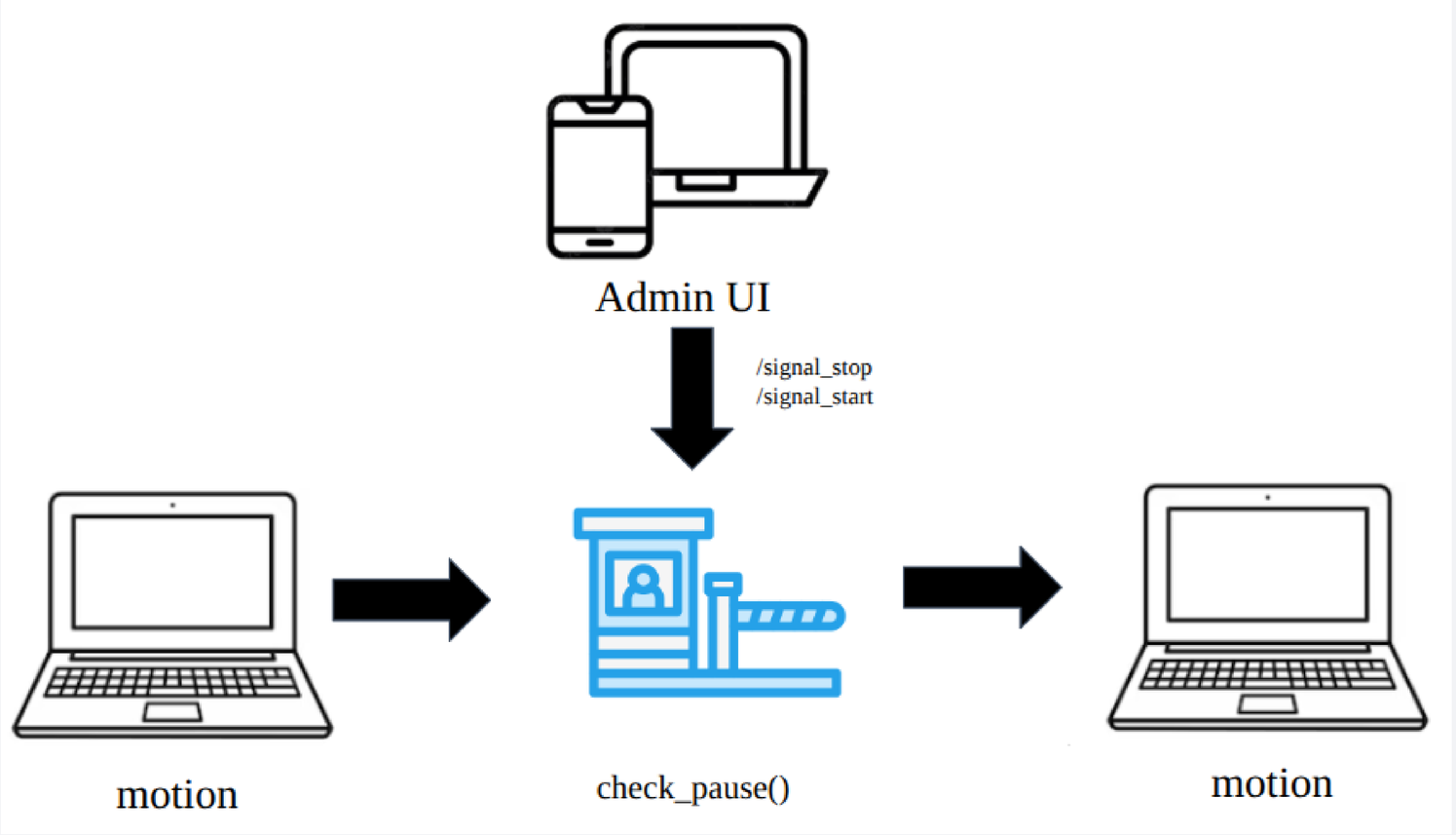
일시정지 & 재개 로직 구현

문제 발생 : 일시정지 토픽 발행 시 전 과정 종료 후 입력



Control

일시정지 & 재개 로직 구현



문제 해결

Check_pause() 사용

/signal_stop 수신 시 is_paused =True

매번 동작 전 Check_pause를 수행하여

is_paused =True인 경우 무한 대기

/signal_start 수신 시 is_paused =False

무한 대기 상태에서 is_paused =False인 경우

다음 동작 재개

Control

일시정지 & 재개 로직 구현

```python
def check_pause(self):
    global is_paused
    from DSR_ROBOT2 |

    while rclpy.ok():
        if is_paused:
            time.sleep(0.5)
        else:
            break
```

```python
def movel2(self, *args, **kwargs):
    self.check_pause()
    from DSR_ROBOT2 import movel
    return movel(*args, **kwargs)

def movej2(self, *args, **kwargs):
    self.check_pause()
    from DSR_ROBOT2 import movej
    return movej(*args, **kwargs)
```

문제 해결

Function Wrapping 사용

movej와 movel 수행 전 check_pause() 수행

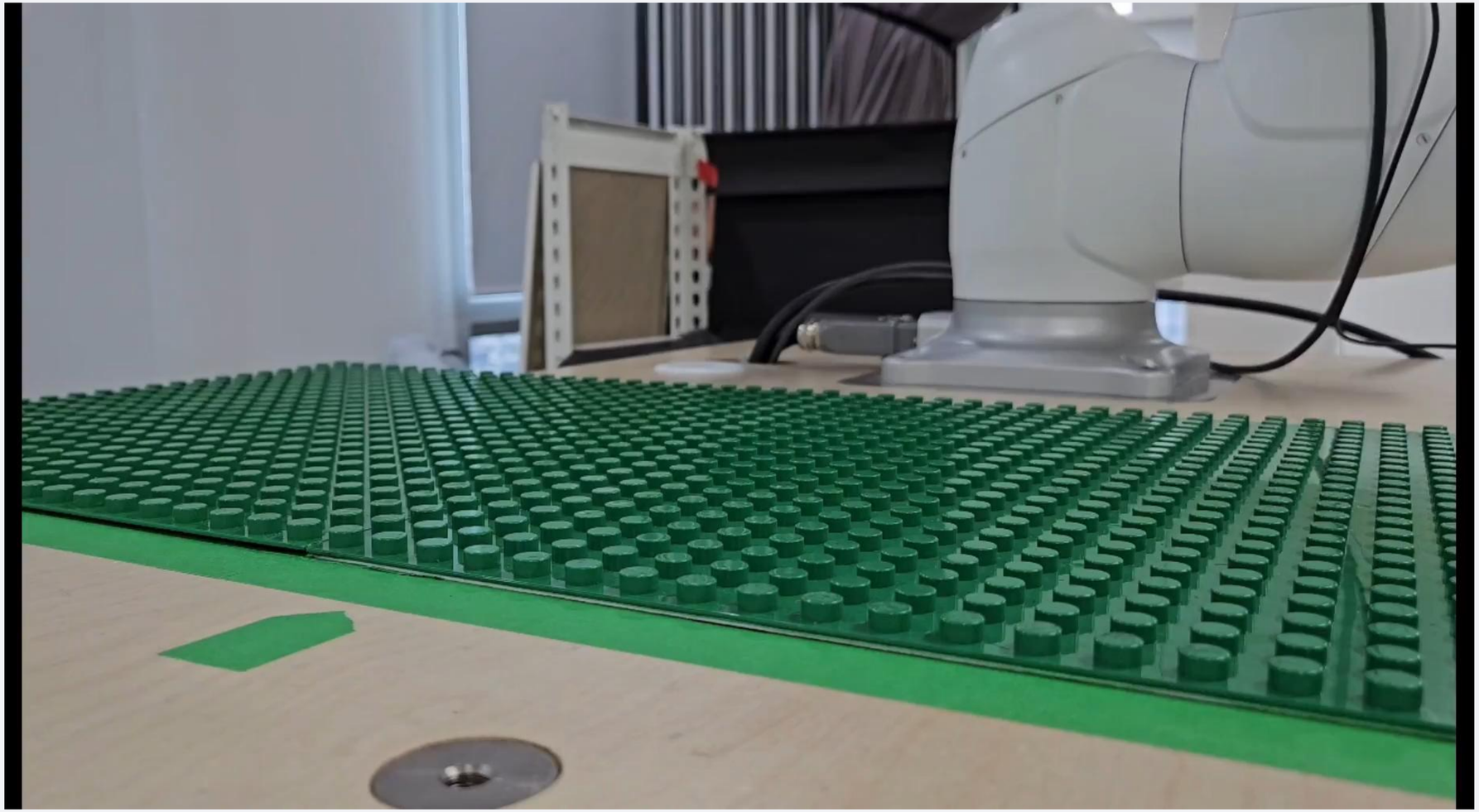
compliance/force 제어 상태에서 정지한 경우
compliance/force해제 후 정지

재개 시 compliance/force도 같이 재개

Control

일시정지 & 재개 로직 구현

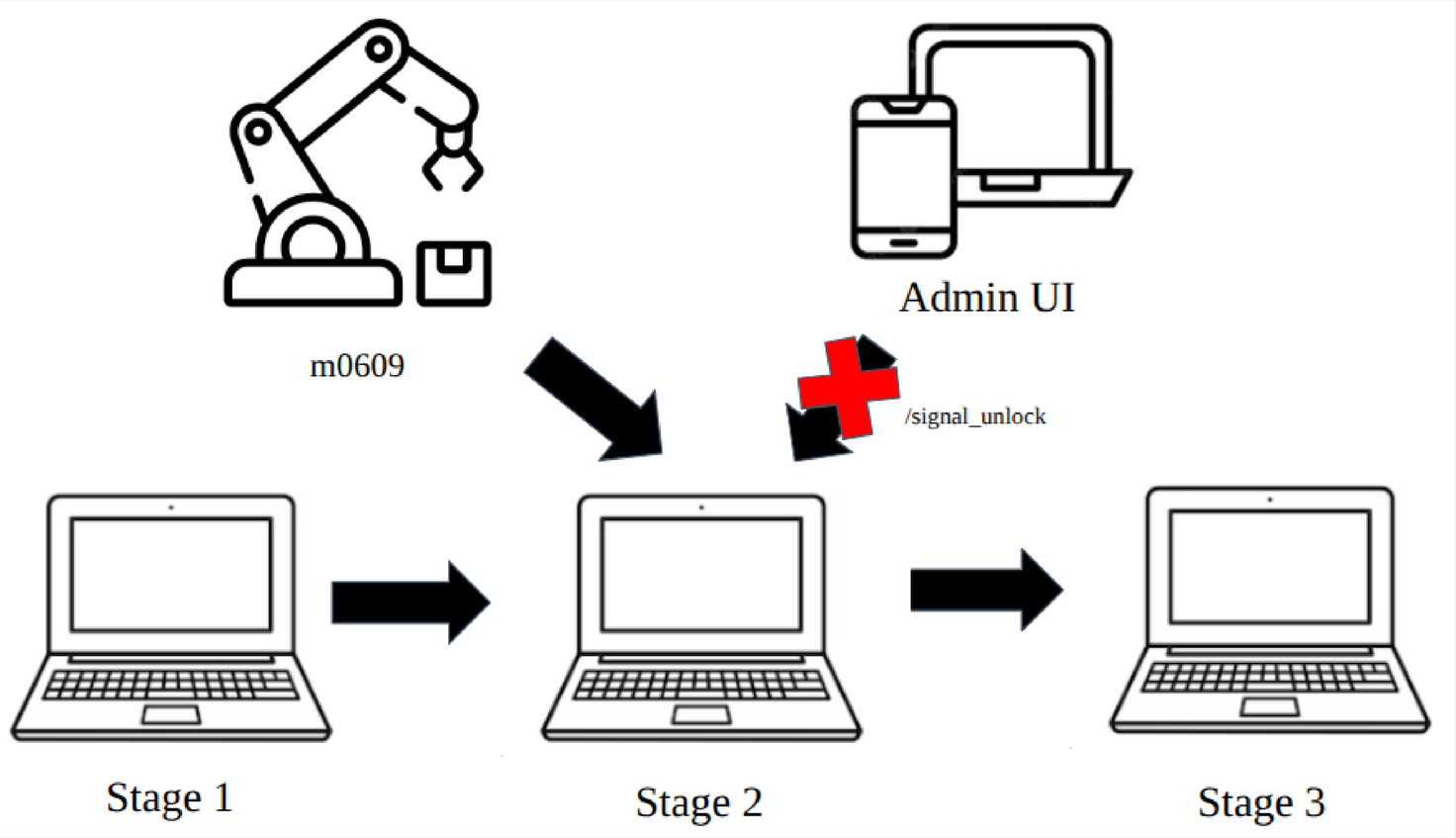
결과 : 일시정지 & 재개 로직 구현



Control

안전복구 로직 구현

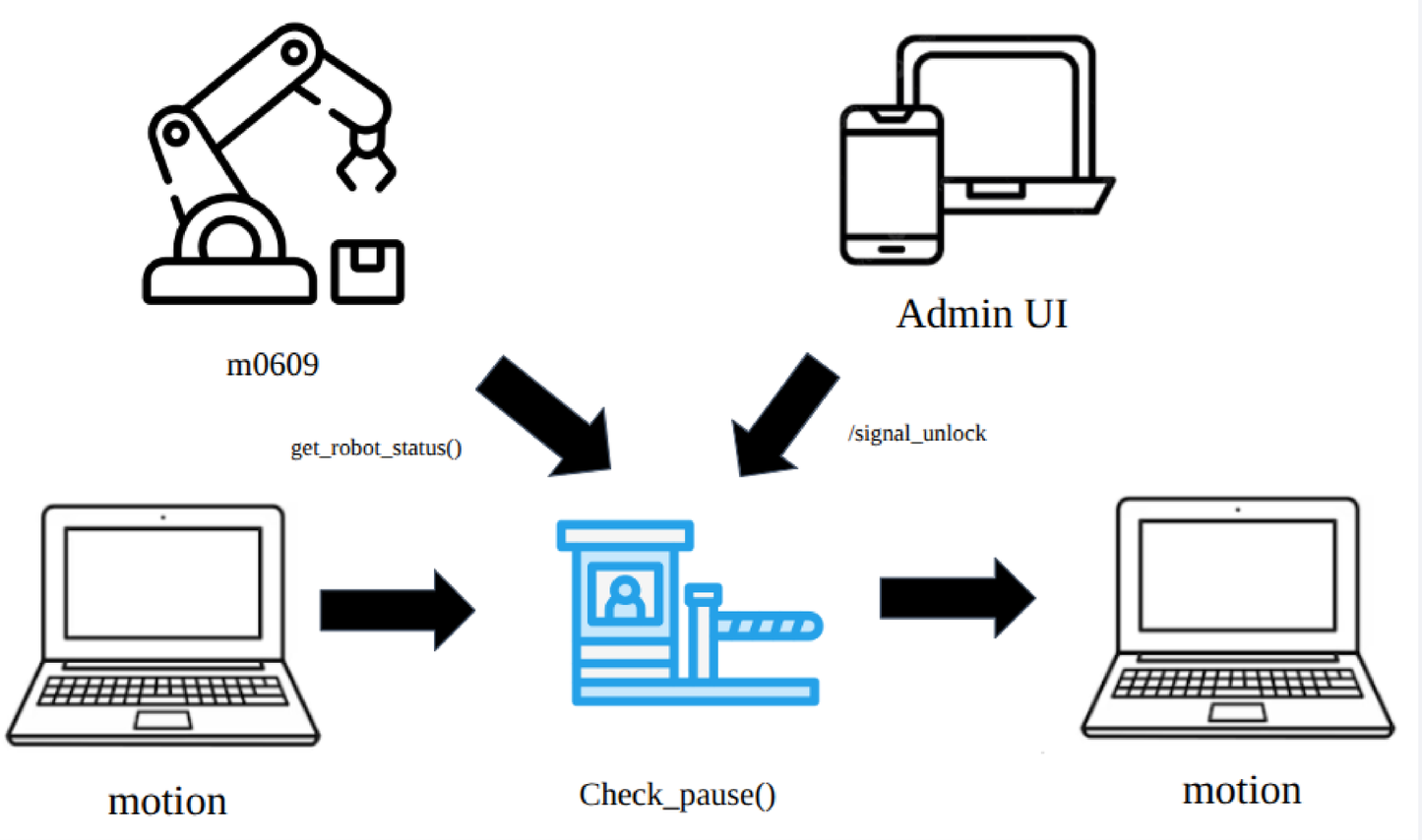
문제 발생 : 일시정지 토픽 발행 시 전 과정 종료 후 입력



Control

안전복구 로직 구현

문제 발생 : 일시정지 토픽 발행 시 전 과정 종료 후 입력



문제 해결

검문소 로직 사용

매번 동작 전 Check_pause를 수행

STATE_SAFE_STOP(안전 정지)
STATE_SAFE_OFF (모터 정지)
상태인 경우 needs_unlock =True

needs_unlock =True인 경우 무한 대기

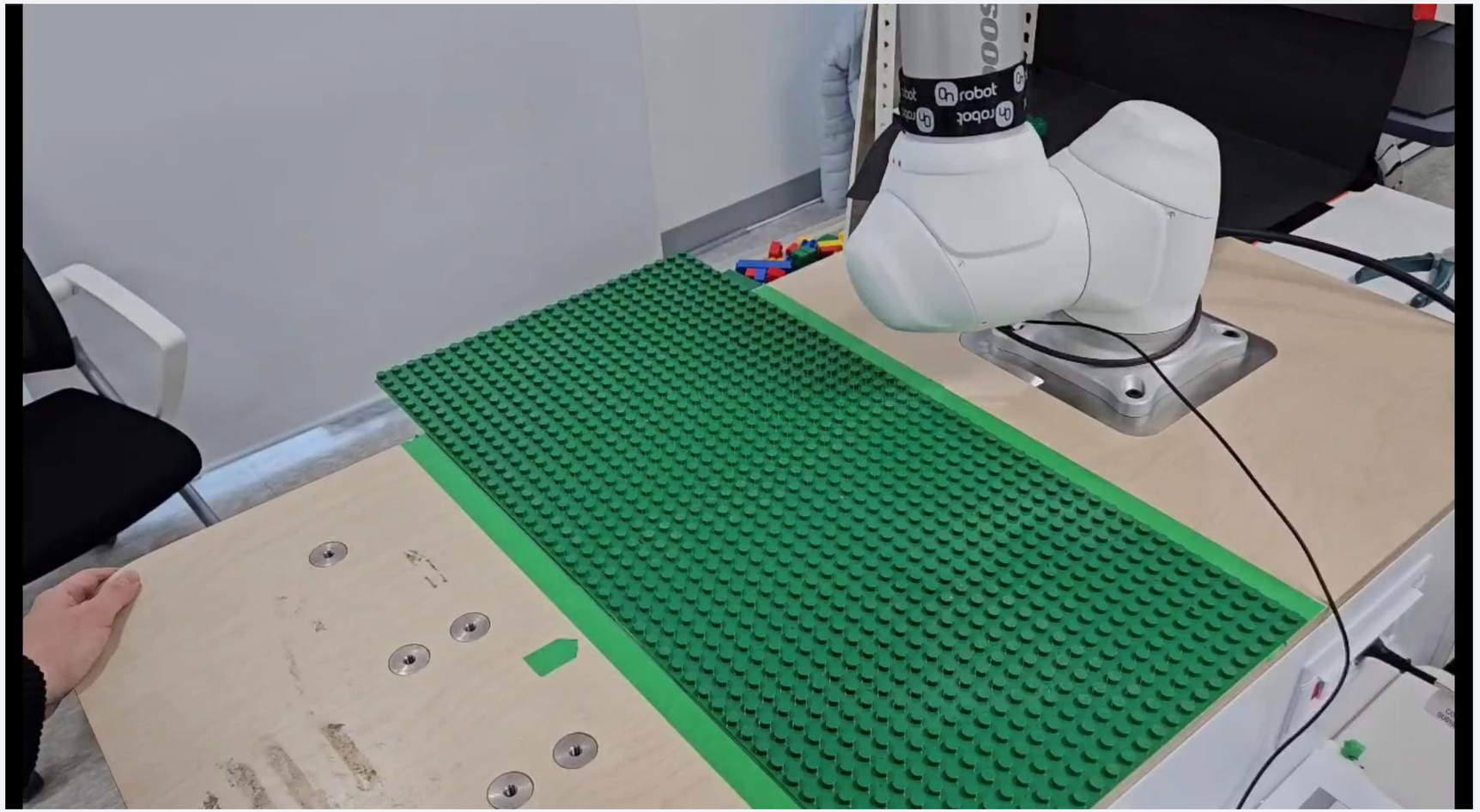
/signal_unlock 수신 시
CONTROL_RESET_SAFE_STOP
(안정정지 해제) 서비스 call

STATE_STANDBY(대기) 상태인 경우
무한 대기 해제 후 다음 동작 실행

Control

안전복구 로직 구현

결과 : 안전 복구 로직 구현



Control

안전복구 로직 구현

결과 : 안전 복구 로직 구현



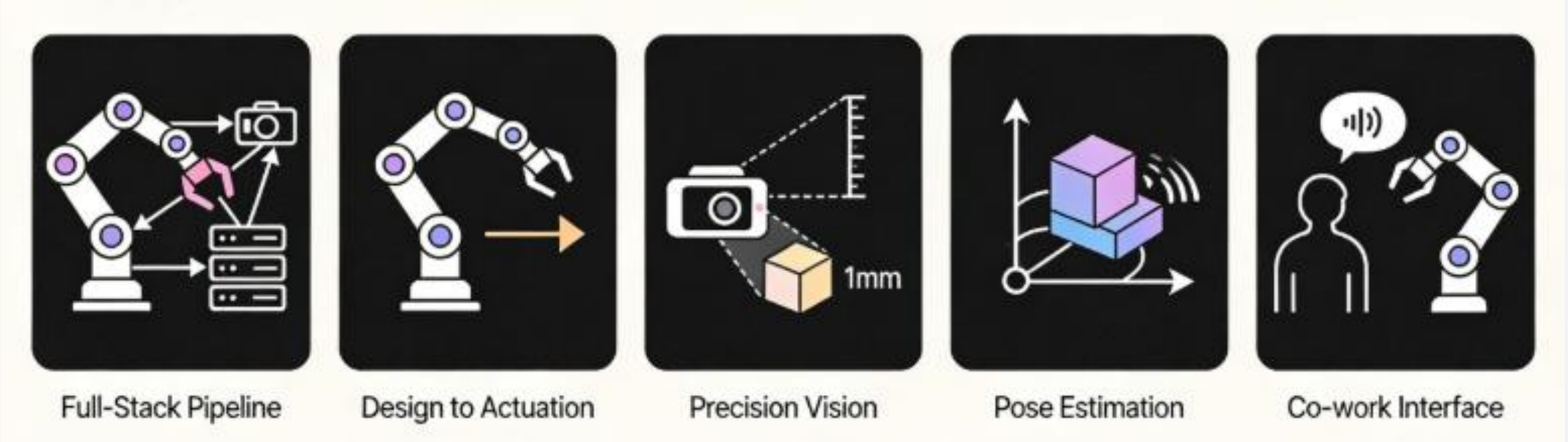
JIUM

05.
자체 평가 의견

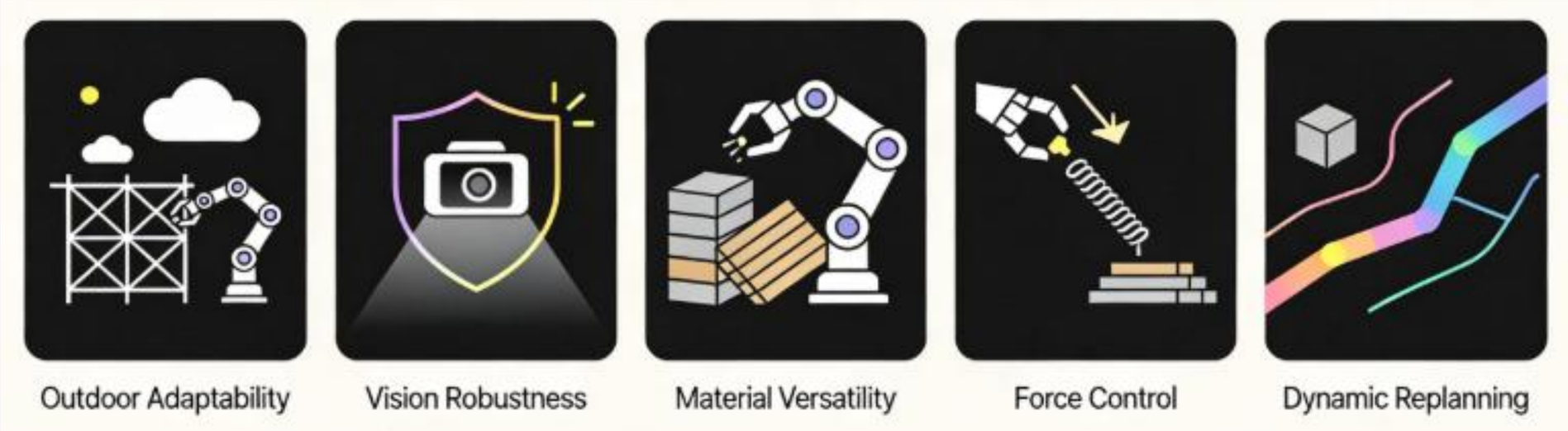
- 프로젝트 주제 및 선정 배경
- 프로젝트 내용
- 활용방안 및 기대 효과



Key Achievements (핵심 성과)



Improvement (개선점 및 향후 과제)



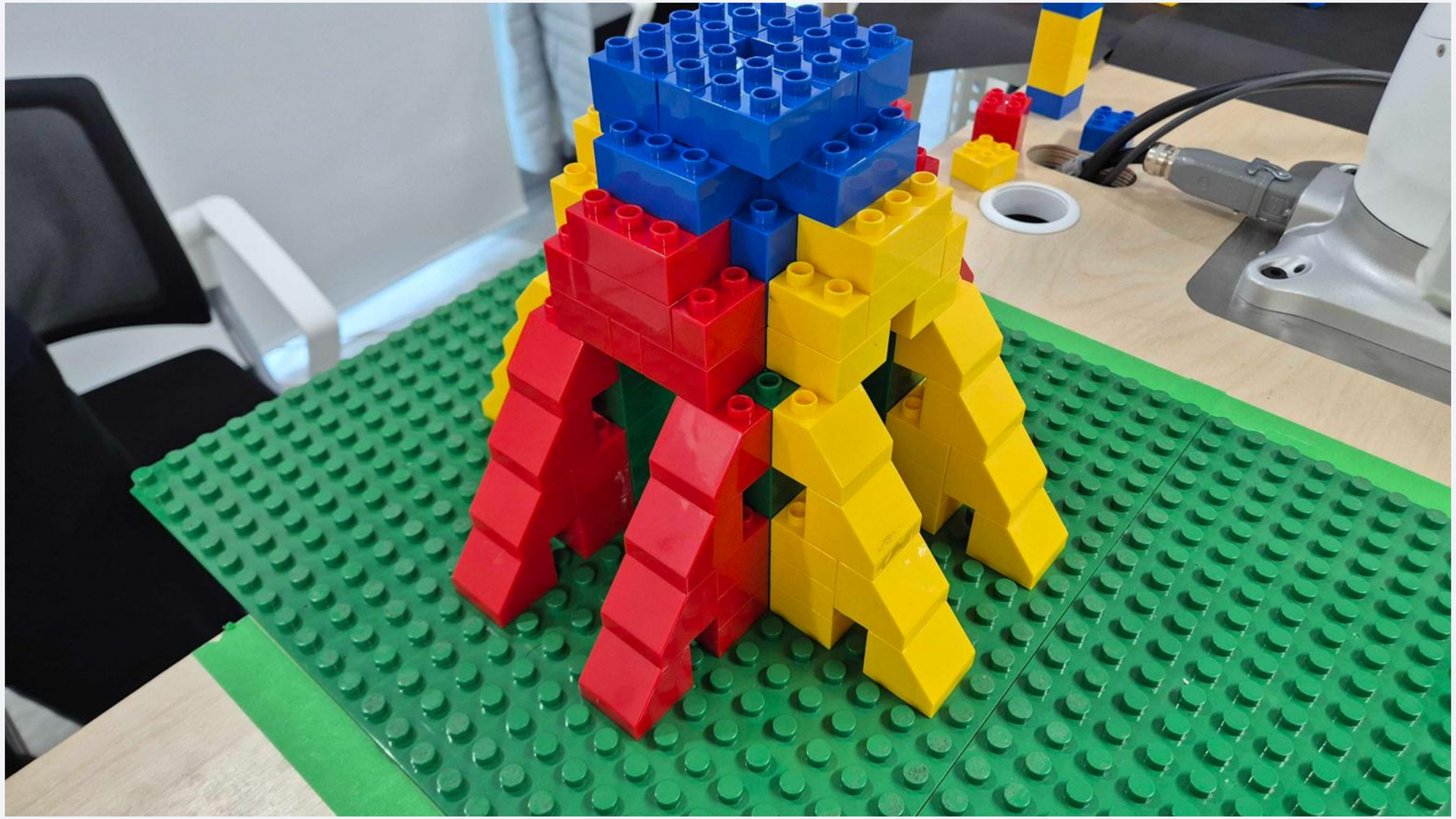
Final construction

Maya Pyramid



Final construction

Maya Pyramid



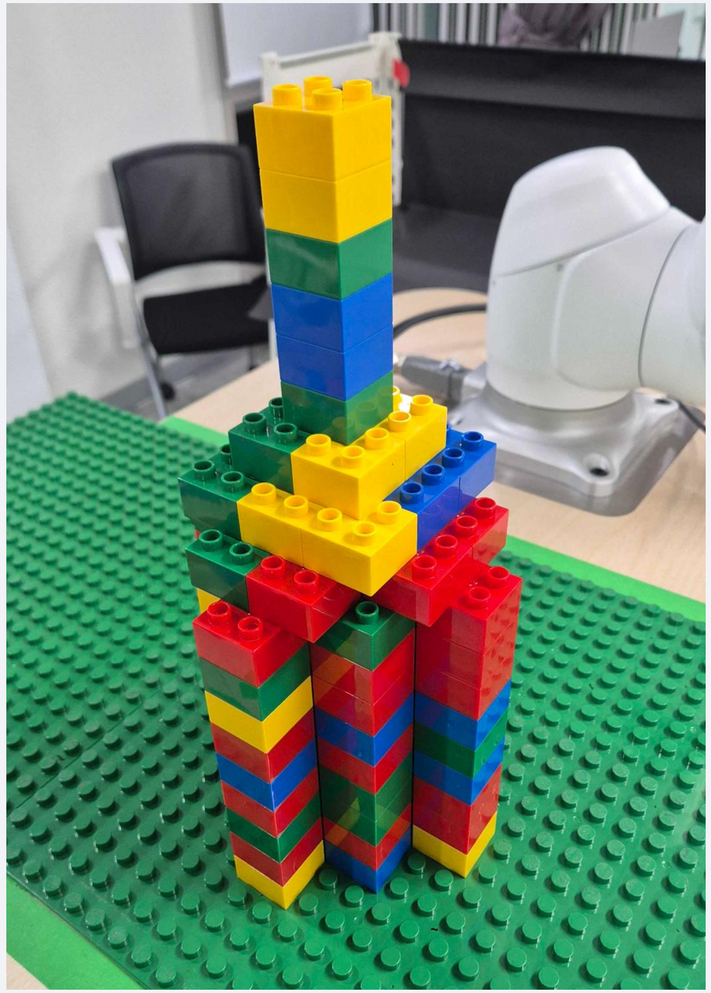
Final construction

Chrysler Building



Final construction

Chrysler Building



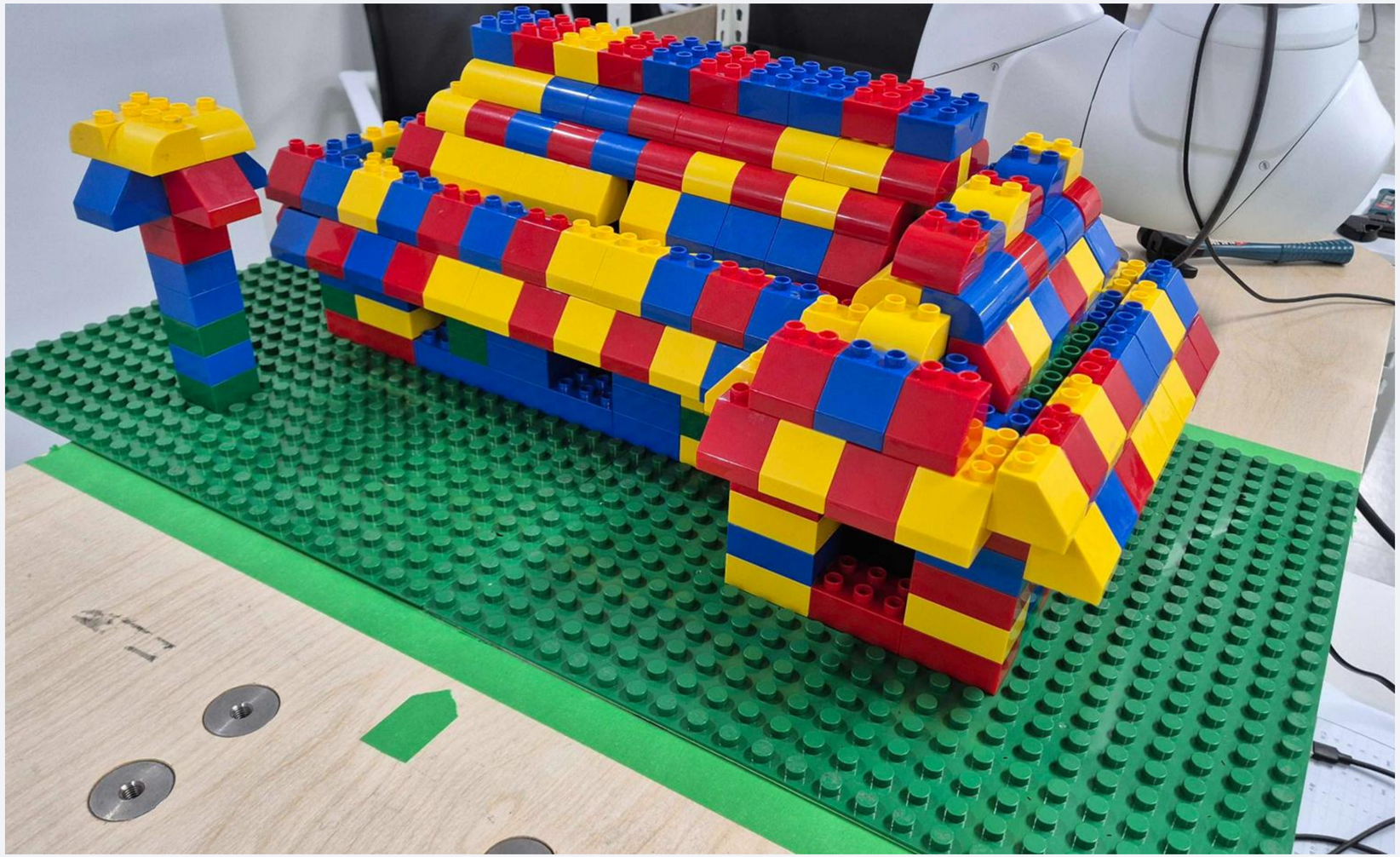
Final construction

Hanok



Final construction

Hanok



JIUM

Thank you

발표 경청해 주셔서 감사합니다.
추가 질문 있으시다면 말씀해 주세요.

Q&A

